C++ 项目式训练手册 · 为人工智能专业大一学生编写

Ways to cpp

你已经学完了变量、数据类型、运算符、输入输出、分支与循环、数组、字符串、结构、指针、函数。

这本手册不想再把这些语法讲一遍。它想带你用手里已有的零件，做出十件真正能跑、能看、能用的东西——然后在做的过程里，把语言知识变成工程直觉。

10	4	30+	0
个完整项目	个循序渐进的阶段	个新知识点	处为了炫技而写的代码

目　　录

00　怎么用这本手册

00　学习地图

00　新知识点总表

01　学生成绩分析器 —— 用向量代替数组

02　英文词频统计器 —— 让文件与字典为你工作

03　康威生命游戏 —— 二维世界与规则的力量

04　迷宫最短路径 —— 广度优先搜索

05　表达式求值器 —— 递归下降分析

06　井字棋与极小极大 —— 会下棋的递归

07　像数学一样的矩阵类 —— 类与运算符重载

08　训练第一个模型：感知机

09　朴素贝叶斯分类器 —— 从数据里学概率

10　曼德博集合与图像 —— 把数学变成产品

A　让代码优雅的十条习惯

B　编译、调试与求助

C　从这里走向 Python 与人工智能

BEFORE YOU START

怎么用这本手册

你说过一句话：语法都学过，但一到算法层面就无从下手。这不是你的问题，这是**所有只做课后题的人都会遇到的坎**。 课后题的形式是「输入 A，输出 B，用一个循环解决」，而真实程序的形式是「有一堆数据，我该怎么摆放它们， 该派哪些函数去处理，出错时谁来兜底」。这两件事用的是同一门语言，却是两种完全不同的思维方式。

所以这本手册的每一个项目，都按照同一套结构来写，你按这个顺序读、按这个顺序做，效果最好：

板块	它是干什么用的
项目背景	先告诉你这件事在真实世界里长什么样。你会知道自己在造什么，而不是在凑一个循环。
本板块新知识点	**每个项目开头都会先列出没学过的内容**。看到不认识的词不要慌，那是接下来要解决的问题的答案。
任务拆解	把一个项目切成 Lv1 / Lv2 / Lv3 三关。**请务必先自己动手写，卡住了再看参考实现。**
参考实现	完整源码 + 真实运行结果。代码可以在 `ways-to-cpp-code/` 里找到，直接编译就能跑。
美在哪里	逐条讲解这段代码为什么这样写更好看、更好改。**这是本手册最想给你的东西。**
常见陷阱 / 进阶挑战	前者帮你少走弯路，后者给你继续往前走的方向。

三条硬规矩

1. **先关掉参考实现，自己写四十分钟。**哪怕写得很丑、编译不过、逻辑全错，那四十分钟的价值也大于直接抄一遍。
2. **每一关都要真的编译、真的运行。**看懂和写出来之间，隔着一次 `error: expected ';'`。
3. **动手改。**把示例数据换成你自己的、把 10 改成 100、把升序改成降序。你对代码的掌控感是改出来的。

编译与运行环境

本手册所有代码都在 C++11 / C++14 标准下编写，任何一个现代编译器都能直接通过。以 MinGW 的 g++ 为例：

```
g++ -std=c++14 -Wall p01_grade_analyzer.cpp -o p01

./p01          # 在 Linux / macOS / Git Bash 下
p01.exe        # 在 Windows 的 cmd 或 PowerShell 下
```

`-Wall` 的意思是「把该提醒我的都提醒我」。初学者最该养成的习惯之一，就是**永远带着它编译**——编译器是你身边最便宜、最诚实的老师。

关于「新知识」的分寸

你希望少碰语言的高级特性，这个尺度我按下面的标准把握：

- **收进来的**：标准库中真正日常的工具（`vector`、`map`、`sort`、文件流、`class` 与运算符重载），以及算法本身的思想（搜索、递归、概率）。它们是「做事要用的」，不是「考试要考的」。
- **挡在外面的**：模板元编程、移动语义与右值引用、智能指针、lambda 的进阶用法、多线程、编译期计算、C++20 概念等。这些是有价值的，但对你当前的阶段而言，投入产出比很低，而且 Python 与 AI 的路线用不上它们。

THE MAP

学习地图

十个项目分成四个阶段。阶段与阶段之间是**依赖关系**，不是并列关系——每一个项目都会用到前面项目教过的东西， 所以请按顺序走。难度用五星制表示，但请记住：**难度高的项目不一定花时间多，难度低的项目也值得做得漂亮。**

阶段	编号	项目	难度	你会获得的能力
第一篇 把数据管起来	01	学生成绩分析器	★★☆☆☆	不再用固定长度的数组，学会用 `vector` 和标准库算法组织一批数据
	02	英文词频统计器	★★☆☆☆	让程序读写真实文件，用 `map` 完成「查表式」的统计
	03	康威生命游戏	★★★☆☆	驾驭二维数据与「按规则演化」的思维，这是所有仿真与元胞自动机的骨架
第二篇 让程序会思考	04	迷宫最短路径	★★★☆☆	第一次真正意义上的「算法」：广度优先搜索与路径回溯
	05	表达式求值器	★★★☆☆	递归下降分析——用三十行代码理解编译器的入口，递归从此不再是「算阶乘」
	06	井字棋与极小极大	★★★★☆	把递归用于「对抗性搜索」，写出一个你赢不了的对手
第三篇 用 C++ 写数学	07	像数学一样的矩阵类	★★★☆☆	定义自己的类型，让 `C = A * B` 直接可写。这是神经网络的第一块砖
	08	感知机	★★★★☆	完整走一遍「造数据—训练—评估—保存参数」的机器学习流程
	09	朴素贝叶斯分类器	★★★★☆	用你亲手写的代码，完成一次真实的文本分类
第四篇 把产品做出来	10	曼德博集合与图像	★★★☆☆	程序不再只输出文字，而是产出一个可以用眼睛看的文件——这就是「落地」

> 建议节奏：一周一个项目，两天读、三天写、两天改。
>
> 十周之后，你手里会有十个能跑的程序，和一套比语法重要得多的东西——**面对一个空白文件时知道从哪下手的底气。**

新知识点总表

下面这张表是本手册的「新东西清单」。左边是知识点，中间告诉你它**为了解决什么问题**才出现， 右边一列是它**在 Python 里的近亲**——你是人工智能专业的学生，这条路迟早要走到 Python 上去， 提前建立这种对应关系，将来迁移会轻松得多。

项目	新知识点	它为什么存在	Python 里的对应物
01	const 与 const 引用	告诉编译器也告诉读者：这个数据到此为止，不许改	约定俗成的常量命名
01	std::vector	可以随时长大的数组，长度不再是编译期就定死的数字	list
01	范围 for / std::sort	不再手写下标循环，也不再手写冒泡排序	for x in xs / sorted()
02	std::map	用「键」直接找到「值」，天生适合数数、查表、建索引	dict
02	文件流 fstream	让数据活过这次运行，能被下一次运行读到	open() / with 语句
02	字符分类函数	把「是不是字母」「转成小写」交给标准库，别自己写范围判断	str.isalpha() / str.lower()
03	二维 vector	表示网格、图像、棋盘、矩阵——一切有行有列的东西	嵌套 list
03	方向数组	把「八个方向」变成一张数据表，而不是八段重复的 if	方向元组列表
03	双缓冲更新	读旧状态、写新状态，避免「改到一半被自己影响」	list.copy() / 新建数组
04	std::queue 与 BFS	一层一层向外扩散，第一次到达即最短	collections.deque
04	std::pair 与前驱表	把两个值绑成一个、并记录「我是从哪来的」，用来复原路径	tuple / 字典记录父节点
05	递归下降分析	用一层层函数准确表达「优先级」这件事	递归下降解析器
05	引用参数共享状态	让几个函数共用同一个「光标」，不用全局变量	把对象传来传去

项目	新知识点	它为什么存在	Python 里的对应物
06	极小极大搜索	在「我走一步、对手走一步」的世界里找最优解	同样的算法，加缓存就是 α-β
06	按值传参	每次递归拿到一份自己的副本，撤销变得免费	切片复制 board[:]
07	class 与封装	把数据和操作它的函数捆在一起，外面只看得到接口	class
07	运算符重载	让自定义类型也能用 + 和 *，代码长得像数学公式	__add__ / __mul__
07	const 成员函数	承诺「这个函数不会改动对象」，编译器替你把关	没有直接对应，靠自觉
08	<random> 随机数引擎	造出可复现的假数据，机器学习的实验离不开它	random / numpy.random
08	训练循环	「预测—比较—修正」重复很多次，这就是学习的本质	for epoch in range(n)
08	参数持久化	把训练结果存成文件，下次不用重训	pickle / np.save
09	概率取对数	把连乘变成连加，避免小概率相乘变成 0	np.log 累加
09	拉普拉斯平滑	没见过的词也要给一点点概率，否则整条判断作废	加一平滑
10	二进制文件输出	把内存里的数字原样写进文件，于是程序能产出「作品」	open(..., 'wb')
10	坐标映射	把屏幕像素坐标翻译成数学坐标，这是可视化的第一步	同样的线性映射

> 不要试图记住这张表。**知识点的价值不在于记住，而在于你在某个时刻真的需要它**——那时的记忆，才是长在身上的。

01 学生成绩分析器

把一堆分数交给数据自己去排队

难度 ★★☆☆☆ | 建议 3-5 小时 | 主线：数据组织 | 阶段一

本板块会用到的新知识点

下面这些你还没学过。不用提前去查，读到需要它们的地方，回来对一眼就行。

- **const 引用参数** 形如 `const std::vector<Student>&`。一个 `&` 让函数不用拷贝整份数据，一个 `const` 保证函数不会偷偷改你的东西。
- **std::vector** 需要 `#include <vector>`。长度可以变的数组。你以前写 `int a[100]`，是猜一个足够大的数；vector 让数组自己长大。
- **范围 for** 形如 `for (const Student& s : students)`。「对 students 里的每一个 s」，不必再写 `i = 0; i < n; ++i`。
- **std::sort 与比较函数** 需要 `#include <algorithm>`。标准库自带的排序，比手写冒泡快得多；你只需告诉它「谁该排前面」。
- **static_cast** 形如 `static_cast<double>(n)`。明确地做一次类型转换，比 C 风格的 `(double)n` 更醒目、更安全。

一、这件事在真实世界里长什么样

每学期期末，老师手里都有一张成绩表。他想要的东西会不断变：今天想知道平均分，明天想排出名次， 后天想看看「这个班是不是两极分化」——那是标准差要回答的问题；再过两天教务又要按等级统计人数。

如果你把这些需求全塞进 `main` 里，写成一段长长的循环，那么每来一个新需求，你都要重新读一遍那段代码， 小心翼翼地不要碰坏别的地方。**这个项目要教你的，就是把「数据」和「对数据的操作」分开。** 分开了之后，加一个需求，只是多写一个函数而已。

二、任务拆解：三关

Lv1

先把数据装起来

定义 `struct Student { std::string name; int score; };`，用你在数组那一章学过的写法， 造 8 个学生的固定数组，打印平均分和最高分的同学。

这一关只用你已经会的东西

，目的是热身。

Lv2

把动作拆成函数

把「平均分」「最高分」「标准差」各写成一个独立函数。函数参数用 `const std::vector<Student>&`，容器改用 `std::vector`。

检验标准：main 里只剩下「准备数据」和「调用函数」两种语句。

Lv3

让它能排名、能评级

用 `std::sort` 把学生按分数从高到低排好，输出一张带序号、姓名、分数、等级的成绩单。 排序规则单独写成一个函数，名字要能读懂。

三、参考实现

ways-to-cpp-code / p01_grade_analyzer.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 01  学生成绩分析器
//  编译：  g++ -std=c++14 -Wall p01_grade_analyzer.cpp -o p01
//  运行：  ./p01        (Windows 下是 p01.exe)
// ============================================================

#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

// ---------- 1. 先描述"数据长什么样" ----------
// 结构体只负责装数据，不负责做事，这是最省心的分工。
struct Student {
    std::string name;
    int         score;
};

// ---------- 2. 再做"数据上的动作"，一个函数只做一件事 ----------

// const 引用：既不拷贝（快），又向读代码的人承诺"我不会改你"
double averageOf(const std::vector<Student>& students) {
    if (students.empty()) return 0.0;
    double sum = 0.0;
    for (const Student& s : students) {   // 范围 for：不用再关心下标
        sum += s.score;
    }
    return sum / static_cast<double>(students.size());
}

// 返回最高分学生的下标；调用者保证容器非空
int indexOfBest(const std::vector<Student>& students) {
    int best = 0;
    for (int i = 1; i < static_cast<int>(students.size()); ++i) {
        if (students[i].score > students[best].score) best = i;
    }
```

```
    return best;
}

// 标准差：衡量"分数有多散"。这就是算法，只不过只有六行。
double deviationOf(const std::vector<Student>& students) {
    if (students.size() < 2) return 0.0;
    const double mean = averageOf(students);
    double acc = 0.0;
    for (const Student& s : students) {
        const double d = s.score - mean;
        acc += d * d;
    }
    return std::sqrt(acc / static_cast<double>(students.size()));
}

// 排序规则单独写成函数，名字就是说它的那句话，比写在调用处好读得多
bool higherScoreFirst(const Student& a, const Student& b) {
    return a.score > b.score;
}

// 一条 if-else 链，其实就是一张查表
std::string gradeOf(int score) {
    if (score >= 90) return "A";
    if (score >= 80) return "B";
    if (score >= 70) return "C";
    if (score >= 60) return "D";
    return "F";
}

void printTable(const std::vector<Student>& students) {
    std::cout << "----- Ranked -----\n";
    std::cout << std::left  << std::setw(4)  << "#"
              << std::left  << std::setw(10) << "Name"
              << std::right << std::setw(6)  << "Score"
              << std::right << std::setw(7)  << "Grade" << "\n";
    for (int i = 0; i < static_cast<int>(students.size()); ++i) {
        const Student& s = students[i];
        std::cout << std::left  << std::setw(4)  << (i + 1)
                  << std::left  << std::setw(10) << s.name
                  << std::right << std::setw(6)  << s.score
                  << std::right << std::setw(7)  << gradeOf(s.score) << "\n";
    }
}

int main() {
    // 演示数据。真实使用时，把这一段换成 cin 读入即可，
    // 上面所有函数一行都不用改 —— 这就是"把数据和逻辑分开"的回报。
    std::vector<Student> students = {
        {"Alice",  92}, {"Bob",    67}, {"Cathy",  85},
        {"David",  58}, {"Eve",   100}, {"Frank",  74},
        {"Grace",  81}, {"Henry",  46}
    };

    std::cout << std::fixed << std::setprecision(2);
```

```
    std::cout << "Students : " << students.size() << "\n";
    std::cout << "Average  : " << averageOf(students) << "\n";
    std::cout << "StdDev   : " << deviationOf(students) << "\n";
    std::cout << "Best     : " << students[indexOfBest(students)].name
              << " (" << students[indexOfBest(students)].score << ")\n\n";

    std::sort(students.begin(), students.end(), higherScoreFirst);
    printTable(students);
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
Students : 8
Average  : 75.38
StdDev   : 16.75
Best     : Eve (100)

----- Ranked -----
#   Name       Score  Grade
1   Eve          100      A
2   Alice         92      A
3   Cathy         85      B
4   Grace         81      B
5   Frank         74      C
6   Bob           67      D
7   David         58      F
8   Henry         46      F
```

> 这段代码美在哪里
>
> 1. **结构体只装数据，函数只做动作。** `Student` 里一行逻辑都没有，`averageOf` 里一行存储都没有。 这种「各自的活各自干」的分工，是后面所有项目的基础。
> 2. **一个 const 引用，同时买到两样东西。** `&` 免去拷贝整份数据的开销，`const` 让「我不会改你」这句话变成编译器能验证的事实。 读代码的人看到这个签名，就再也不用担心 `averageOf` 会动他的数据。
> 3. **排序规则有了名字。** 比较一下这两种写法：`sort(v.begin(), v.end(), [](auto& a, auto& b){return a.score>b.score;})` 和 `sort(v.begin(), v.end(), higherScoreFirst)`。 后者读起来就是一句英文，而且这个规则可以在别处复用、可以单独测试。
> 4. **函数是「纯」的。** `averageOf` 除了传入的容器，不依赖任何外部状态。 这样的函数可以被随手调用一万次，结果永远一样——这是可靠代码最重要的性质。
> 5. **换数据源，不用改逻辑。** 把 `main` 里那段演示数据换成 `cin` 读入， 上面五个函数一行都不用动。**这就是「解耦」在真实项目里的回报。**

常见陷阱

- `students.size()` 返回的是无符号整数，直接和 `int i` 比较，编译器会警告「有符号与无符号比较」。 本手册统一写成 `static_cast<int>(students.size())`。
- 比较函数里千万别写 `>=`。`std::sort` 要求「严格弱序」， 写 `>=` 会在某些数据下越界崩溃——这是个非常经典的坑。
- 空容器求平均会除以 0。所以 `averageOf` 第一行就先挡住 `empty()` 的情况。
- 浮点输出不加 `std::setprecision` 会打印出 `75.375` 这样的默认六位有效数字，也可能是 `1e+06`。要显示小数，记得 `<iomanip>` 里的 `std::fixed << std::setprecision(2)`。

进阶挑战

- 让程序从 `scores.txt` 读入成绩（每行「姓名 分数」）。读文件的方法在项目 02 会讲，你可以先自己试试。
- 再加一套排序：「按姓名升序」。你会发现排序规则函数可以随便换，其余代码纹丝不动。
- 求出中位数，并比较它和平均分的差距。差距大，说明分数分布是偏的——这是真实数据分析里最常用的第一眼判断。
- 统计每个等级有几个人，并且用 `#` 画一个横向条形图。想想看：这个「等级 → 人数」的对应关系，用什么容器存最自然？

---

02 英文词频统计器

让文件和字典替你做苦力

难度 ★★☆☆☆ | 建议 3-5 小时 | 主线：查表与持久化 | 阶段一

本板块会用到的新知识点

这个项目会引入两个你以后几乎天天要用的东西：map 和文件。

- **std::map** 需要 `#include <map>`。「键 → 值」的对应表。 它的下标 `counter[word]` 有一个非常贴心的性质：**键不存在时会自动创建一个值为 0 的条目，** 所以「数数」这个动作被压缩成了一行 `++counter[w]`。
- **文件流** `std::ifstream` 读、`std::ofstream` 写，需要 `#include <fstream>`。 让数据活过这一次运行。用法和 `cin` / `cout` 几乎一模一样，只是把「屏幕」换成了「文件」。
- **std::getline** 需要 `#include <string>`。一次读一整行。注意：`cin >> s` 遇到空格就停， 而 `getline` 会连空格一起读进来——读文本时这个区别至关重要。
- **std::isalpha / std::tolower** 需要 `#include <cctype>`。判断「是不是字母」、把大写变小写。 标准库早就写好了，别自己写 `if (c >= 'A' && c <= 'Z')`。
- **typedef / using 起别名** 形如 `typedef std::map<std::string,int> Counter;`。 把一长串类型名缩写成一个词，读代码时脑子轻松很多。
- **std::pair** 需要 `#include <utility>`。把「词」和「次数」捆成一件东西， 这样它们才能一起被排序。`.first` 是第一个，`.second` 是第二个。

一、这件事在真实世界里长什么样

「数一篇文章里每个词出现了多少次」听起来像课堂练习，但它其实是搜索引擎、文本分类、语言模型的第一步。 你要做的搜索引擎，需要知道哪些词重要；你要做的垃圾邮件过滤器（项目 09），需要知道哪些词可疑； 你要做的情感分析，需要知道「好」和「差」各出现了几次。

而「查表」这件事的通用解法，就是 `map`。**学会它之后，你会发现一大类问题都变成了同一道题。**

二、任务拆解：三关

Lv1

先把文本切干净

写一个函数，输入一整段文本，输出一串「干净的单词」：去掉标点、统一小写、连续字母才算一个词。

先别管统计

，把切分这一个动作做对、做稳。*提示：边扫边攒，遇到非字母就把攒好的词交出去。*

Lv2

数数，然后按次数排名

用 `map` 统计词频（这一步只有三行）。然后你会发现 `map` 是按「键」排的， 而你要按「值」排——所以把它搬进 `vector<std::pair<std::string,int>>`，再用项目 01 学过的 `std::sort`。

Lv3

让它读真正的文件

把文本存成 `sample.txt`，用 `ifstream` + `getline` 一次一行读进来。 再加一点贴心设计：文件不存在时给出友好提示，而不是让程序默默输出一堆 0。

三、参考实现

ways-to-cpp-code / p02_word_count.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 02  英文词频统计器
//  编译：  g++ -std=c++14 -Wall p02_word_count.cpp -o p02
//  运行：  ./p02
//  说明：  第一次运行会在当前目录生成 sample.txt，然后统计它。
// ============================================================

#include <algorithm>
#include <cctype>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <utility>
#include <vector>

// 词 -> 次数。map 的下标访问会自动建表，天生适合数数。
typedef std::map<std::string, int> Counter;
typedef std::pair<std::string, int> WordCount;

// ---------- 一、把一坨文本切成一个个小写单词 ----------
std::vector<std::string> splitWords(const std::string& text) {
    std::vector<std::string> words;
    std::string current;
    for (char raw : text) {
        const unsigned char ch = static_cast<unsigned char>(raw);
        if (std::isalpha(ch)) {
            current += static_cast<char>(std::tolower(ch));  // 统一成小写
        } else if (!current.empty()) {
            words.push_back(current);
            current.clear();
        }
    }
    if (!current.empty()) words.push_back(current);   // 别忘了最后一个
    return words;
}

// ---------- 二、数数本身就是一行 ----------
```

```cpp
Counter countWords(const std::vector<std::string>& words) {
    Counter counter;
    for (const std::string& w : words) {
        ++counter[w];              // 第一次见到它是 0 -> 1，之后一直加
    }
    return counter;
}

// ---------- 三、排序规则：次数多的在前，同次数按字典序，保证输出稳定 ----------
bool moreFrequentFirst(const WordCount& a, const WordCount& b) {
    if (a.second != b.second) return a.second > b.second;
    return a.first < b.first;
}

// ---------- 四、文件读写：先把素材落盘，再读回来 ----------
void saveSampleIfMissing(const std::string& path) {
    std::ifstream probe(path);
    if (probe.good()) return;          // 已经存在就不覆盖，免得改坏你的文件

    std::ofstream fout(path);
    fout << "It was the best of times, it was the worst of times.\n"
            "It was the age of wisdom, it was the age of foolishness.\n"
            "It was the season of light, it was the season of darkness.\n"
            "We had everything before us, we had nothing before us.\n";
}

std::string readAll(const std::string& path) {
    std::ifstream fin(path);
    if (!fin) return std::string();    // 空串表示读失败

    std::string all, line;
    while (std::getline(fin, line)) {  // 一行一行读，不会吞掉换行
        all += line;
        all += ' ';
    }
    return all;
}

int main() {
    const std::string path = "sample.txt";
    saveSampleIfMissing(path);

    const std::string text = readAll(path);
    if (text.empty()) {
        std::cerr << "Cannot read " << path << "\n";
        return 1;
    }

    const std::vector<std::string> words = splitWords(text);
    const Counter counter = countWords(words);

    // map 天生按 key 有序，但我们要按"次数"排，于是搬进 vector 再排
    std::vector<WordCount> ranked(counter.begin(), counter.end());
    std::sort(ranked.begin(), ranked.end(), moreFrequentFirst);
```

```
    std::cout << "File      : " << path << "\n";
    std::cout << "Words     : " << words.size() << "\n";
    std::cout << "Distinct  : " << counter.size() << "\n\n";
    std::cout << "----- Top 10 -----\n";

    const int top = (ranked.size() < 10) ? static_cast<int>(ranked.size()) : 10;
    for (int i = 0; i < top; ++i) {
        std::cout << std::left  << std::setw(14) << ranked[i].first
                  << std::right << std::setw(3)  << ranked[i].second << "  "
                  << std::string(ranked[i].second, '#') << "\n";
    }
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
File      : sample.txt
Words     : 46
Distinct  : 19

----- Top 10 -----
it             6  ######
of             6  ######
the            6  ######
was            6  ######
age            2  ##
before         2  ##
had            2  ##
season         2  ##
times          2  ##
us             2  ##
```

这段代码美在哪里

1. **「数数」被压缩成了一行。** 换成数组的写法，你要先判断这个词出现过没有、出现在哪个位置、再累加。`++counter[w]` 之所以能成立，是因为 `map` 的下标访问会在键不存在时自动建一个 0。 **好的数据结构，能把算法「吃掉」。**
2. **函数按照「一件事」来切。** `splitWords` 只负责切，`countWords` 只负责数，`readAll` 只负责读。三个函数分别可以单独测试，也分别可以单独替换。
3. **排序规则考虑了「平局」。** 只按次数排序时，次数相同的词顺序是不确定的，每次运行可能不一样。 加一句「次数相同按字典序」，输出就完全稳定了。**稳定性是工程素养，不是小事。**
4. **把字符统一成小写再统计。** 否则 `It` 和 `it` 会被算成两个词。 真实数据永远是脏的，清洗要在最前面做。
5. **写文件之前先检查文件是否存在。** 一个看起来无关紧要的小判断，避免了「把用户自己的文件覆盖掉」这种灾难。

常见陷阱

- `cin >> word` **会在空格处停下，** `getline` **不会。** 混用这两者时，前一个留下的换行符会被后一个立刻读走，导致「少读一行」。这是初学者最常撞的墙。
- `std::tolower` 的参数必须是 `unsigned char` 转过来的值。 直接传负数字符在某些平台上会越界崩溃，所以要写 `std::tolower(static_cast<unsigned char>(c))`。
- `counter[w]` 只能用在「非 const」的 map 上。如果你把 map 声明为 `const`， 就必须用 `.find()` 来查（项目 09 会这么做）。
- 忘了 `current.clear()` 或者忘了最后那个词——切词函数里最容易漏掉的就是这两个点。

进阶挑战

- 支持中文文本。中文字符是「多字节」的，一个汉字在 UTF-8 里占三个字节——你会直观地看到「字符」和「字节」的区别， 以及为什么处理中文比英文麻烦。
- 统计「二元组」：把相邻两个词当成一个单位来数（比如 `of the`）。 这是语言模型最原始的形态，也只需要把 map 的键换成 pair。
- 加一个「停用词表」：`the`、`a`、`of` 这类词不统计。你会发现前几名立刻换人了。
- 让程序接受命令行参数指定文件名。C++ 里 `main(int argc, char* argv[])` 就是干这个的。

---

03 康威生命游戏

三个简洁的规则，一个会自己生长的世界

难度 ★★★☆☆ | 建议 4-6 小时 | 主线：二维数据与规则 | 阶段一

本板块会用到的新知识点

这个项目会教你两件影响深远的事：二维数据怎么摆，以及怎样把「重复的代码」变成「数据」。

- **二维 vector** 形如 `std::vector<std::vector<int>>`。「装着一行行数组的数组」。网格、棋盘、图像、矩阵，全都是这个形状。
- **using 类型别名** `using Grid = std::vector<std::vector<int>>;` 写完这一行，后面所有函数签名里就只出现 `Grid` 一个词，读起来清爽得多。
- **方向数组** 形如 `const int DR[8] = {-1,-1,-1,0,0,1,1,1};` 把「八个方向」变成两张静态表。八个方向不再需要八段 if，而是一个循环。这是算法里最常见的一次「用数据消灭代码」。
- **双缓冲更新** 读旧表 `current`，写新表 `next`。所有格子必须**同时**更新。如果你直接在原表上改，后面算的格子会读到已经被改过的邻居，结果全乱。
- **const 语义的深入** `liveNeighbors(const Grid& g, ...)` 传进来只读的用 `const Grid&`，要改的用 `Grid&`。看函数签名就知道它会不会动我的数据，这是 C++ 独有的表达力。

一、这件事在真实世界里长什么样

1970 年，数学家约翰·康威设计了一个「零玩家游戏」：一个方格世界里，每个格子要么活着要么死了，下一代怎么变，只看它周围八个邻居里有几个活人。

规则只有三条：邻居少于 2 个，孤单而死；邻居是 2 或 3 个，继续活着；邻居多于 3 个，拥挤而死；死格子周围正好 3 个活人，就复活。

就这么三条规则，却能长出滑翔机、振荡子、甚至能计算的逻辑门。**这个项目真正的价值不在游戏本身，而在于你第一次写出「按规则演化」的程序结构**——它和森林火灾模拟、疫情传播模型、元胞自动机、乃至卷积神经网络里的滑窗操作，是同一副骨架。

二、任务拆解：三关

Lv1 **先把世界画出来**

造一个 14 行 34 列的二维网格，把「滑翔机」这个经典图案摆进去，写一个函数把它打印出来。

先不管演化，能看见就是胜利。

Lv2 **数邻居，然后算下一代**

写 `liveNeighbors()`，注意

四条边界都要检查越界

。 然后写 `nextGeneration()`：它接收旧网格，返回一张全新的网格。 *关键：不要在原网格上直接改。*

Lv3

让世界动起来

把「打印 → 演化」放进一个循环，连续输出若干代。

观察滑翔机每四代向右下移动一格

—— 当你亲眼看到它动起来的时候，你才算真正理解了这个程序。

三、参考实现

ways-to-cpp-code / p03_game_of_life.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 03  康威生命游戏
//  编译:  g++ -std=c++14 -Wall p03_game_of_life.cpp -o p03
//  运行:  ./p03
// ============================================================

#include <iostream>
#include <string>
#include <vector>

// 给类型起个好名字：以后看到 Grid 就知道是"细胞的方阵"
using Grid = std::vector<std::vector<int> >;

const int ROWS = 14;
const int COLS = 34;

// ---------- 一、造一块空棋盘 ----------
Grid makeGrid(int rows, int cols) {
    return Grid(rows, std::vector<int>(cols, 0));
}

// ---------- 二、八个方向，写成一张表 ----------
// 把"八个方向"变成数据而不是八段 if，这是算法里非常常用的一步。
const int DR[8] = {-1, -1, -1,  0, 0,  1, 1, 1};
const int DC[8] = {-1,  0,  1, -1, 1, -1, 0, 1};

int liveNeighbors(const Grid& g, int r, int c) {
    int count = 0;
    for (int k = 0; k < 8; ++k) {
        const int nr = r + DR[k];
        const int nc = c + DC[k];
        if (nr >= 0 && nr < ROWS && nc >= 0 && nc < COLS) {  // 越界就跳过
            count += g[nr][nc];
        }
    }
    return count;
```

```
}

// ---------- 三、下一代：读旧格子，写新格子 ----------
// 关键在于"整体同时更新"。若在原地改，后面的格子会读到已经变过的邻居。
Grid nextGeneration(const Grid& current) {
    Grid next = makeGrid(ROWS, COLS);
    for (int r = 0; r < ROWS; ++r) {
        for (int c = 0; c < COLS; ++c) {
            const int n = liveNeighbors(current, r, c);
            if (current[r][c] == 1) {
                next[r][c] = (n == 2 || n == 3) ? 1 : 0;   // 活细胞：太挤或太孤单都会死
            } else {
                next[r][c] = (n == 3) ? 1 : 0;             // 死细胞：正好三个邻居就复活
            }
        }
    }
    return next;
}

// ---------- 四、打印 ----------
void printGrid(const Grid& g, int generation) {
    std::cout << "generation " << generation << "\n";
    std::cout << "+" << std::string(COLS, '-') << "+\n";
    for (int r = 0; r < ROWS; ++r) {
        std::cout << "|";
        for (int c = 0; c < COLS; ++c) {
            std::cout << (g[r][c] ? '#' : '.');
        }
        std::cout << "|\n";
    }
    std::cout << "+" << std::string(COLS, '-') << "+\n\n";
}

// ---------- 五、一块著名的初始图案：滑翔机 ----------
void placeGlider(Grid& g, int top, int left) {
    const int pattern[3][3] = {
        {0, 1, 0},
        {0, 0, 1},
        {1, 1, 1}
    };
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            g[top + r][left + c] = pattern[r][c];
        }
    }
}

int main() {
    Grid grid = makeGrid(ROWS, COLS);
    placeGlider(grid, 1, 1);

    for (int generation = 0; generation <= 4; ++generation) {
        printGrid(grid, generation);
        grid = nextGeneration(grid);
```

```
    }
    return 0;
}
```

实际运行结果（节选：第 0 代与第 4 代）

```
generation 0
+--------------------------------+
|................................|
|..#.............................|
|...#............................|
|.###............................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
+--------------------------------+


        ( ..... 中间第 1 ~ 3 代略 ..... )


generation 4
+--------------------------------+
|................................|
|................................|
|...#............................|
|....#...........................|
|..###............................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
|................................|
+--------------------------------+
```

> 这段代码美在哪里
>
> 1. **八个方向变成了两张表。** 不用方向数组的写法，`liveNeighbors` 里会有八段几乎一样的 `if (r-1 >= 0 && c-1 >= 0) count += g[r-1][c-1];`。 用了方向数组之后只剩五行。**当你想复制粘贴第三遍的时候，就该停下来想想：这段东西能不能变成数据？**
> 2. **「读旧、写新」是一次重要的思维升级。** 让 `nextGeneration` 接收一个 const 引用、 返回一个全新的 Grid，从函数签名上就杜绝了「边算边改」的错误。函数的形状本身在保护你。
> 3. **规则写成了和规则本身一样的形状。** 看这两个三目运算符：`next[r][c] = (n == 2 || n == 3) ? 1 : 0;`。它读起来几乎就是「邻居是二或三就活」这句话本身。 **代码和意图之间没有多余的翻译层，这就是优雅。**
> 4. **参数名说明了用途。** `placeGlider(Grid& g, int top, int left)` 里的 top 和 left， 比 x、y 或 a、b 好懂得多。命名的功夫，就是替未来的自己省时间。

常见陷阱

- **最常见的错误：边遍历边修改。** 症状是图案会在几代之内莫名其妙地死光。原因就是后面的格子读到了已经被改写的邻居。 解法只有一个：写进另一张表。
- 二维 vector 的初始化不能写 `Grid g(rows, cols)`，那是「rows 个值为 cols 的元素」。 正确的是 `Grid(rows, std::vector<int>(cols, 0))`。
- 边界忘记检查会读到别的行——C++ 不会报错，只会算出荒唐的结果。这类错误最耗时间，所以**越界判断要写得像条件反射**。
- 想让它「动」起来需要 `<thread>` 和 `<chrono>` 里的睡眠函数。 这两个头文件在某些老版本的 Windows 编译环境下支持不佳，所以本手册的版本故意不加入动画—— **先保证代码在任何机器上都能跑，再考虑锦上添花。**

进阶挑战

- 把边界从「死墙」改成「环绕」：第 0 行的上面是最后一行。只需要改一处越界判断，但世界的行为会完全不同。
- 实现几个经典图案：闪烁器（Blinker）、方块（Block）、滑翔机枪（Gosper Glider Gun）。后者会源源不断地吐出滑翔机。
- 让用户用鼠标……目前还不行，但你可以让用户用键盘输入坐标来手动放置细胞，做一个「编辑器」。
- 统计每一代的活细胞数，输出成一个序列。观察它有没有周期性——**这就是最简单的「数据分析」**。
- 想想看：如果把「数邻居」换成「加权求和」，把「存活规则」换成「和大于阈值就激活」，你手里就有一个卷积层了。

---

04 迷宫最短路径

像水波一样，一圈一圈地扩散

难度 ★★★☆☆ | 建议 5 - 7 小时 | 主线：广度优先搜索 | 阶段二

本板块会用到的新知识点

从这里开始，你正式进入「算法」。好消息是：这个算法只有二十行。

- **std::queue 队列** 需要 `#include <queue>`。「先进先出」的容器。用 `push()` 从队尾放进，用 `front()` 看队首、`pop()` 弹出队首。**它和栈（后进先出）的区别，就是「一圈一圈扩散」和「一条路走到黑」的区别。**
- **std::pair** 需要 `#include <utility>`。把两个值捆成一件东西，比如一个坐标 `Cell(3, 5)`。它可以直接用 `==` 比较，所以写 `if (cur == goal)` 就够了。
- **广度优先搜索（BFS）** 这是思想，不是语法：**把起点放进队列，每次取出一个，把它的所有「下一步」放到队尾。**因为队列是先进先出的，所以所有距离为 1 的格子会先被处理完，才轮到距离为 2 的。于是——**第一次到达终点时走的路，一定是最短的。**
- **visited / dist 表** 形如 `std::vector<std::vector<int>> dist`。一张和地图同样大的表，记录「到每个格子最少要几步」。它顺便还兼任了「来没来过」的标记。
- **前驱表与路径回溯** 代码里的 `parent`。BFS 只告诉你「最短是 128 步」，不告诉你「怎么走」。于是我们顺手记下「我是从哪个格子来的」，最后从终点一路倒推回起点，路就出来了。

一、这件事在真实世界里长什么样

你手机里的导航软件规划的路线、游戏里 NPC 找到你的路径、社交网络里「你和某个人隔了几度」、网络数据包经过了几台路由器——这些问题的数学形式完全一样：**一张图，从一点到另一点，最少几步。**

解决办法朴素得让人意外：**从起点开始一圈一圈往外扩散，扩散到终点为止。**这不就是往平静的水面丢一颗石子吗？最先漫到岸边的那个水波，走的一定是最直的路线。

二、任务拆解：三关

Lv1 **把地图读进来，把规则写清楚**

用 `std::vector<std::string>` 存地图（`#` 是墙，`.` 是路）。写一个打印函数，再写一个「这个格子能不能走」的判断。

这一关只是把问题描述清楚，还没开始搜索。

Lv2 **算出最短步数**

建一个和地图等大的 `dist` 表，全部填 `-1` 表示「没去过」。 起点入队、距离记 0，然后开始循环：取出队首，把四个方向上「能走且没去过」的邻格记上「当前距离 + 1」并入队。

能打印出 128 这个数字，这一关就过了。

Lv3

把路画出来

再加一张 `parent` 表，入队时顺便记下「我是从哪来的」。 搜索结束后，从终点一路顺着 parent 往回走，把沿途的 `.` 改成 `*`。

三、参考实现

ways-to-cpp-code / p04_maze_bfs.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 04  迷宫最短路径（广度优先搜索）
//  编译：  g++ -std=c++14 -Wall p04_maze_bfs.cpp -o p04
//  运行：  ./p04
// ============================================================

#include <iostream>
#include <queue>
#include <string>
#include <utility>
#include <vector>

using Grid = std::vector<std::string>;
using Cell = std::pair<int, int>;          // (行, 列)

const int DR[4] = {-1, 1, 0, 0};           // 上 下 左 右
const int DC[4] = {0, 0, -1, 1};

void printGrid(const Grid& g) {
    for (const std::string& row : g) {
        std::cout << "  " << row << "\n";
    }
}

// 广度优先搜索：像往水里丢石头，一圈一圈向外扩散。
// 第一次到达终点时走过的圈数，就是最短路的长度。
int shortestPath(Grid& grid, Cell start, Cell goal) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());

    std::vector<std::vector<int> > dist(rows, std::vector<int>(cols, -1));
    std::vector<std::vector<Cell> > parent(rows, std::vector<Cell>(cols, Cell(-1, -1)));

    std::queue<Cell> frontier;
    frontier.push(start);
    dist[start.first][start.second] = 0;

    while (!frontier.empty()) {
```

```
        const Cell cur = frontier.front();   // 取出队首（先进先出）
        frontier.pop();

        if (cur == goal) break;              // 到终点了，可以收工

        for (int k = 0; k < 4; ++k) {
            const int nr = cur.first + DR[k];
            const int nc = cur.second + DC[k];

            const bool inside = (nr >= 0 && nr < rows && nc >= 0 && nc < cols);
            if (!inside) continue;                        // 撞墙外
            if (grid[nr][nc] == '#') continue;            // 撞墙
            if (dist[nr][nc] != -1) continue;             // 已经来过（这才是 BFS 不绕圈的关键）

            dist[nr][nc] = dist[cur.first][cur.second] + 1;
            parent[nr][nc] = cur;
            frontier.push(Cell(nr, nc));
        }
    }

    if (dist[goal.first][goal.second] == -1) return -1;   // 走不到

    // 从终点顺着 parent 一路回溯到起点，把路画在图上
    Cell step = goal;
    while (!(step == start)) {
        if (grid[step.first][step.second] == '.') grid[step.first][step.second] = '*';
        step = parent[step.first][step.second];
    }
    return dist[goal.first][goal.second];
}

int main() {
    Grid grid = {
        "#################################",
        "#S..#.......#.....#..........#",
        "###.######.#.###.#.########.###.#",
        "#.#.#.....#...#.#...#.....#...#",
        "#.#.#.########.#.###.#.######.###",
        "#.#.#.#.#.....#...#...#...#...#",
        "#.#.#.#.#.######.#.######.#.###.#",
        "#...#.#...#...#.#.....#.#.....#",
        "#.###.#.###.#.########.#.########",
        "#.....#.....#.........#......E#",
        "#################################"
    };

    const Cell start(1, 1);
    const Cell goal(9, 29);

    std::cout << "Maze:\n";
    printGrid(grid);
    std::cout << "\n";

    const int steps = shortestPath(grid, start, goal);
```

```
    if (steps < 0) {
        std::cout << "No path found.\n";
    } else {
        std::cout << "Shortest distance : " << steps << " steps\n\n";
        std::cout << "One shortest path ('*'):\n";
        printGrid(grid);
    }
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
Maze:
  ##################################
  #S..#.......#.....#..........#
  ###.#####.#.###.#.#######.###.#
  #.#.#.....#...#.#...#.....#...#
  #.#.#.#######.#.###.#.#####.###
  #.#.#.#.#.....#...#...#...#...#
  #.#.#.#.#.#####.#.#####.#.###.#
  #...#.#...#...#.#.....#.#.....#
  #.###.#.###.#.#######.#.#######
  #.....#.....#.........#......E#
  ##################################

Shortest distance : 128 steps

One shortest path ('*'):
  ##################################
  #S**#....***#..***#......*****#
  ###*######*#*####*#*#######*####*#
  #.#*#*****#***#*#***#*****#***#
  #.#*#*#######*#*####*#*#####*###
  #.#*#*#.#*#*****#***#***#***#***#
  #.#*#*#.#*#*****#.#*#*****#*#***#
  #***#*#***#***#.#*#*****#*#*****#
  #*###*#*###*#*#######*#*#######
  #*****#*****#*********#******E#
  ##################################
```

这段代码美在哪里

1. **「最短」这个要求，被策略本身免费满足了。** 代码里没有任何一行在比较「哪条路更短」。 它只是在扩散。扩散的顺序天然保证了先到的一定最短。**这就是算法的美：不是算得更快，而是想得更对。**
2. **一张表干了两件事。** `dist[r][c]` 是 `-1`， 既表示「没访问过」，也表示「到不了」。一个哨兵值同时承担两种含义，于是 `if (dist[nr][nc] != -1) continue;` 这一行就同时挡住了「重复访问」和「绕圈」。
3. **「算距离」和「还原路径」是分开的。** BFS 只负责填 `dist`， 回溯只负责读 `parent`。两段逻辑互不干扰，各自都能单独看懂。**把「求解」和「呈现」分开，是工程上极重要的一刀。**
4. **方向数组思想在项目 03 出现过，这里又出现了。** 四个方向写成 `DR/DC` 两张表， 于是「上、下、左、右」变成一次循环。**好思想会被反复用到——这正是它们值得被记住的原因。**
5. **传引用还是传 const 引用，取决于你要不要改它。** `shortestPath(Grid& grid, ...)` 用的是普通引用，因为它要在地图上画 `*`； 而 `printGrid(const Grid& g)` 用 const，因为它只看不改。**两个函数签名放在一起，就把意图说完了。**

常见陷阱

- **标记「已访问」必须在入队时，不能等到出队时。** 如果在出队时才标记，同一个格子会被好几个邻居重复入队， 队列会膨胀几倍，代码依然能跑对，但慢得莫名其妙。这是个很隐蔽的坑。
- `queue` 取队首用 `front()`，不是 `back()`。 用错就变成了深度优先——结果不再是「一圈一圈」，最短性立刻失效。
- `pop()` 的返回值是 `void`，它只负责扔掉。 想取值必须先 `front()` 再 `pop()`。
- 网格的行是 `std::string`，所以 `grid[r][c]` 取出来的是 `char`，比较时要用 `'#'` 而不是 `"#"`。
- 起点就等于终点、或者终点被墙包死——这两种情况都要能优雅处理，而不是崩溃。

进阶挑战

- 把所有最短路径全部打印出来。你会发现从「记录一个父亲」变成「记录所有父亲」，再改成深度优先输出。
- 允许斜着走（八个方向）。最短步数会变吗？如果斜走一步和直走一步「代价」不一样呢？
- 给每个格子加上不同的通行代价（草地慢、大路快），这时 BFS 就不够用了，你需要 **Dijkstra 算法** —— 它和 BFS 只差一个数据结构：把队列换成优先队列。
- 再加一个「离终点还有多远」的估计值，优先搜索离终点近的方向，这就是 **A* 算法**，游戏工业里的寻路基石。

---

05 表达式求值器

用函数的调用层次，表达运算的优先级

难度 ★★★☆☆ | 建议 5-8 小时 | 主线：递归下降分析 | 阶段二

本板块会用到的新知识点

你会第一次遇到「递归不只是算阶乘」这件事。这个项目的代码量比前几个都少，但思想密度最高。

- **递归下降分析** 这是本项目的灵魂。核心想法只有一句：**用函数的调用层次来表达运算的优先级。** 加法函数会去调用乘法函数，所以乘法天生就比加法先算——优先级不再是一张需要查的表，而是程序的形状本身。
- **函数前置声明** 形如在文件上方写 `double parseTerm(const std::string&, size_t&);` C++ 要求「先声明后使用」。而这三个函数互相调用，谁也无法排在第一个，所以必须先声明、后定义。
- **引用参数共享状态** `size_t& pos` 三个函数需要共享同一个「读到哪了」的光标。用引用而不是全局变量，既不破坏封装，也不需要写一个类。
- **`std::atof`** 需要 `#include <cstdlib>`。 把一段字符串转成 `double`。有了它，我们不必手写「数位乘十累加」。
- **`size_t`** 无符号整数类型，专门用来表示「长度」和「下标」。标准库容器的下标就是它。

一、这件事在真实世界里长什么样

你在计算器里敲下 `1 + 2 * 3`，它回给你 `7`。为什么不是 `9`？因为你默认了乘号先算。 可计算机只会从左往右读字符，它凭什么知道该先算哪一段？

这个问题是编译器技术的起点。JavaScript 引擎要理解你的代码、Excel 要计算公式、SQL 要解析查询语句， 走的都是同一条路：**先把表达式理解成一棵树，再算这棵树。**

而写这棵树最短的写法，叫递归下降——**三十行代码，三个函数，就能把一个完整的四则运算表达式吃透。**

二、任务拆解：三关

Lv1 只会加法

写一个函数，只处理「数字 + 数字 + 数字」这种形式。

先跑通「读一个数字」和「吃掉一个加号」这两件事，不要贪多。这一步的目的是想清楚「光标 pos 是共享的」。

Lv2 加入乘除，让优先级自然出现

把原来的函数拆成两个：一个只管加减，一个只管乘除。让「加减函数」在需要操作数时去调用「乘除函数」。

体会一下：你一行优先级判断都没写，但 `1 + 2 * 3` 已经等于 7 了。

Lv3 **加入括号和负号**

再加最里层的第三个函数：它负责「读一个数字」「处理括号」「处理负号」。 遇到左括号时，它递归调用最外层的加减函数——

于是括号里的世界重新开始，任何复杂度都被这一行递归抹平了。

三、参考实现

ways-to-cpp-code / p05_calculator.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 05  表达式求值器（递归下降）
//  编译：  g++ -std=c++14 -Wall p05_calculator.cpp -o p05
//  运行：  ./p05
//  支持：  + - * / ( ) 与小数
// ============================================================

#include <cctype>
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <string>

// 三个函数互相调用，所以先在这里"报个到"（函数声明）
double parseExpression(const std::string& s, size_t& pos);
double parseTerm(const std::string& s, size_t& pos);
double parseFactor(const std::string& s, size_t& pos);

void skipSpaces(const std::string& s, size_t& pos) {
    while (pos < s.size() && std::isspace(static_cast<unsigned char>(s[pos]))) ++pos;
}

// 最外层：只认加号减号。位置 pos 用引用传递，于是三个函数共享同一个"光标"。
double parseExpression(const std::string& s, size_t& pos) {
    double value = parseTerm(s, pos);
    while (true) {
        skipSpaces(s, pos);
        if (pos < s.size() && (s[pos] == '+' || s[pos] == '-')) {
            const char op = s[pos];
            ++pos;                                  // 吃掉这个运算符
            const double rhs = parseTerm(s, pos);
            value = (op == '+') ? value + rhs : value - rhs;
        } else {
            return value;
        }
```

```
    }
}

// 中间层：只认乘号除号。它的优先级天生比加减高，因为它被更里层调用。
double parseTerm(const std::string& s, size_t& pos) {
    double value = parseFactor(s, pos);
    while (true) {
        skipSpaces(s, pos);
        if (pos < s.size() && (s[pos] == '*' || s[pos] == '/')) {
            const char op = s[pos];
            ++pos;
            const double rhs = parseFactor(s, pos);
            value = (op == '*') ? value * rhs : value / rhs;
        } else {
            return value;
        }
    }
}

// 最里层：数字、括号、负号
double parseFactor(const std::string& s, size_t& pos) {
    skipSpaces(s, pos);

    if (pos < s.size() && s[pos] == '(') {
        ++pos;                                          // 跳过 '('
        const double value = parseExpression(s, pos); // 括号里重新从最外层开始
        skipSpaces(s, pos);
        if (pos < s.size() && s[pos] == ')') ++pos;   // 跳过 ')'
        return value;
    }

    if (pos < s.size() && (s[pos] == '-' || s[pos] == '+')) {
        const bool negative = (s[pos] == '-');
        ++pos;
        const double value = parseFactor(s, pos);     // 允许 --3、-(1+2) 这种写法
        return negative ? -value : value;
    }

    // 读一个数字：整数部分.小数部分
    size_t begin = pos;
    while (pos < s.size() && (std::isdigit(static_cast<unsigned char>(s[pos])) || s[pos] == '.')) {
        ++pos;
    }
    if (begin == pos) return 0.0;                       // 没读到数字，交给上层收尾
    return std::atof(s.substr(begin, pos - begin).c_str());
}

double evaluate(const std::string& expression) {
    size_t pos = 0;
    return parseExpression(expression, pos);
}

int main() {
    const std::string tests[] = {
```

```
        "1 + 2 * 3",
        "(1 + 2) * 3",
        "2 * (3 + 4) - 5",
        "10 / 4",
        "-3 + 10",
        "1 + 2 * (3 + 4 * (5 - 2))",
        "((2 + 3) * (7 - 4)) / 3"
    };

    std::cout << std::fixed << std::setprecision(4);
    std::cout << "----- Expression Evaluator -----\n";
    for (const std::string& t : tests) {
        std::cout << std::left << std::setw(28) << t
                  << " = " << evaluate(t) << "\n";
    }
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
----- Expression Evaluator -----
1 + 2 * 3                    = 7.0000
(1 + 2) * 3                  = 9.0000
2 * (3 + 4) - 5              = 9.0000
10 / 4                       = 2.5000
-3 + 10                      = 7.0000
1 + 2 * (3 + 4 * (5 - 2))    = 31.0000
((2 + 3) * (7 - 4)) / 3      = 5.0000
```

> **这段代码美在哪里**
>
> 1. **优先级变成了结构，而不是规则。** 传统写法要先做「中缀转后缀」，还要维护一个运算符栈、 查一张优先级表。而这里，**优先级就是「谁调用谁」**：加减在顶层、乘除在中间、括号和数字在最里面。 越靠里，越先被算出来。代码的层次结构，直接对应了数学的结构。
> 2. **一个引用参数替代了一个类。** 三个函数共享 `size_t& pos`， 没有全局变量、没有对象、没有成员函数。对于这么小的一个问题，这是最轻的表达方式。 **「够用就好」本身就是一种优雅。**
> 3. **三个函数，每个都短到一屏能看完。** 而且它们的名字就是它们的职责：`parseExpression` 管整个表达式， `parseTerm` 管一个项，`parseFactor` 管一个因子。 **能一句话说清的函数，才配拥有一个好名字。**
> 4. **递归在这里不是炫技，而是唯一的自然写法。** 括号可以套括号，套多少层事先并不知道—— 这正好是递归天生要解决的问题。
> 5. **「吃掉一个字符」这个动作被写得非常明确。** `const char op = s[pos]; ++pos;` 先把记下来，再前进。**把「读」和「移动」分开写，能避免绝大多数「少读一个字符」的 bug。**

常见陷阱

- **忘记 `++pos` 会导致死循环。** 函数不再前进，`while` 就永远转下去。 第一次写的时候几乎一定会遇到，遇到之后你会永远记住它。
- 括号不匹配时，这个简版实现会安静地返回一个错误结果，而不是报错。 **「出错时大声说出来」是工程代码的基本原则**——这里留给你在进阶挑战里补上。
- 三个函数互相调用，**前置声明一个都不能少**，而且声明和定义的签名必须完全一致，否则链接时会报「未定义」。
- 这里全程用 `double`，所以 `10 / 4` 得到 `2.5`。 如果换成 `int`，得到的就是 `2`——整数除法截断是初学者永恒的坑。

进阶挑战

- 加上乘方 `^`。注意它是**右结合**的：`2^3^2` 应该是 `2^9 = 512` 而不是 `8^2 = 64`。 你会发现自己需要把某个 `while` 改成递归调用。
- 支持变量 `x`，并用项目 02 学过的 `map<std::string,double>` 存变量表。 **做完这一步，你手里就有了一个「公式引擎」**——Excel 的一小部分。
- 加入函数调用：`sin(0)`、`max(3, 5)`。你需要处理逗号分隔的多个参数。
- 把「解析」和「计算」彻底分开：先解析成一棵树（结构体 + 指针就是天然的树），再写一个 `evaluate` 去遍历它。 **这一步做完，你就摸到了编译器的门。**

---

06 井字棋与极小极大

写一个你永远赢不了的对手

难度 ★★★★☆ | 建议 6-9 小时 | 主线：对抗性搜索 | 阶段二

本板块会用到的新知识点

这是第二阶段的高潮。递归在这里第一次用来「思考」，而不只是用来「遍历」。

- **极小极大搜索（Minimax）** 思想：**我走一步时，假设我选最好的；对手走一步时，假设他选对我最坏的。** 递归下去，每个局面都会得到一个分数，我选分数最高的那一手。这套逻辑适用于一切「有对手」的场景，也是早期国际象棋程序的基础。
- **按值传参** 形如 `int minimax(std::vector<char> board, ...)`，注意 `board` 前面没有 `&`。 每次递归都拿到棋盘的一份独立副本。于是「试走一步」和「撤销这一步」变成了天然的动作——**函数返回时副本自动消失，外面的棋盘毫发无损。**这是用一点点拷贝开销买来的、极干净的一段递归。
- **`std::numeric_limits`** 需要 `#include <limits>`。 `std::numeric_limits<int>::min()` 就是 `int` 能表示的最小值。 求最大值时把初值设为它，比硬编码一个 `-999999` 更安全、也更说明意图。
- **`std::min` / `std::max`** 需要 `#include <algorithm>`。 极小极大的核心操作就是取最小或取最大，标准库直接给你了。
- **递归中的「试—评—撤」模式** 这是回溯算法的通用骨架，八皇后、数独、全排列，用的都是它。

一、这件事在真实世界里长什么样

井字棋只有九个格子，规则简单到小孩都会玩。但正因为简单，它成了「机器如何做决策」最好的教具： **它小到可以穷举所有未来，又大到不能一眼看穿。**

一九五零年代，程序第一次在井字棋上做到了不败。从那里出发，加上更聪明的剪枝、更准确的局面评估，才有了后来的跳棋程序、国际象棋的深蓝，直到 AlphaGo。**你接下来写下的这二十行递归，是那整条路的起点。**

二、任务拆解：三关

Lv1

先把规则写清楚

用 `std::vector<char>` 表示九宫格。写三个函数：

打印棋盘

、

判断谁赢了

（八条连线，用一张表存起来）、

判断是否下满

。

这一关没有递归，全是基本功。

Lv2 **做一个「随便下」的对手**

写一个函数，在所有空位里随便挑一个落子。

先让它能陪你下完一整局

—— 你会立刻发现：随便下的 AI 输得很难看。这个落差，就是第三关的动力。

Lv3 **把「随便」换成「算尽」**

写 `minimax(board, player, depth)`： 如果已经分出胜负，返回一个带深度的分数；否则轮到谁走，就把每个空位都试一遍，递归拿到分数， 我方取最大、对方取最小。最后用 `bestMove` 在根节点挑出最好的那一手。

三、参考实现

ways-to-cpp-code / p06_tictactoe.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 06  井字棋与极小极大搜索
//  编译：  g++ -std=c++14 -Wall p06_tictactoe.cpp -o p06
//  运行：  ./p06
// ============================================================

#include <algorithm>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

const char EMPTY = '.';
const char HUMAN = 'X';   // 先手
const char AI    = 'O';

const int LINES[8][3] = {
    {0, 1, 2}, {3, 4, 5}, {6, 7, 8},   // 三行
    {0, 3, 6}, {1, 4, 7}, {2, 5, 8},   // 三列
    {0, 4, 8}, {2, 4, 6}               // 两条对角线
};

char winnerOf(const std::vector<char>& b) {
    for (int i = 0; i < 8; ++i) {
        const int a = LINES[i][0], m = LINES[i][1], c = LINES[i][2];
        if (b[a] != EMPTY && b[a] == b[m] && b[m] == b[c]) return b[a];
    }
    return EMPTY;                       // 没人赢
}

bool boardFull(const std::vector<char>& b) {
```

```
    for (char c : b) {
        if (c == EMPTY) return false;
    }
    return true;
}

void printBoard(const std::vector<char>& b) {
    for (int r = 0; r < 3; ++r) {
        std::cout << "  ";
        for (int c = 0; c < 3; ++c) {
            std::cout << b[r * 3 + c];
            if (c < 2) std::cout << " | ";
        }
        std::cout << "\n";
        if (r < 2) std::cout << " ---+---+---\n";
    }
}

// ---------- 极小极大搜索 ----------
// board 是"按值传递"的：每次递归都拿到一份自己的副本，
// 于是我们可以在里面随便改，返回时外面的世界毫发无损。
// 这是用拷贝换来的、极其干净的一段递归。
int minimax(std::vector<char> board, char player, int depth) {
    const char w = winnerOf(board);
    if (w == AI)    return 10 - depth;      // 赢得越早，分数越高
    if (w == HUMAN) return depth - 10;      // 输得越晚，分数越接近 0
    if (boardFull(board)) return 0;         // 平局

    const char next = (player == AI) ? HUMAN : AI;
    int best = (player == AI) ? std::numeric_limits<int>::min()
                              : std::numeric_limits<int>::max();

    for (int i = 0; i < 9; ++i) {
        if (board[i] != EMPTY) continue;

        board[i] = player;                          // 试着走一步
        const int score = minimax(board, next, depth + 1);
        board[i] = EMPTY;                           // 撤销这一步

        if (player == AI) best = std::max(best, score);   // 我尽力拿高分
        else              best = std::min(best, score);   // 对手尽力压低我的分
    }
    return best;
}

// 在所有空位里挑一个"最坏情况下的最好结果"
int bestMove(const std::vector<char>& board, char player, int& outScore) {
    const char next = (player == AI) ? HUMAN : AI;
    int bestIndex = -1;
    int best = (player == AI) ? std::numeric_limits<int>::min()
                              : std::numeric_limits<int>::max();

    for (int i = 0; i < 9; ++i) {
        if (board[i] != EMPTY) continue;
```

```
        std::vector<char> trial = board;
        trial[i] = player;
        const int score = minimax(trial, next, 1);
        const bool better = (player == AI) ? (score > best) : (score < best);
        if (better) {
            best = score;
            bestIndex = i;
        }
    }
    outScore = best;
    return bestIndex;
}

int main() {
    std::cout << "----- Demo 1 : must block -----\n";
    std::vector<char> mid = {
        'X', '.', '.',
        '.', 'X', '.',
        'O', '.', '.'
    };
    printBoard(mid);
    int score = 0;
    const int move = bestMove(mid, AI, score);
    std::cout << "AI ('O') chooses cell " << move
              << " (row " << (move / 3 + 1) << ", col " << (move % 3 + 1) << ")"
              << ", evaluated score = " << score << "\n\n";

    std::cout << "----- Demo 2 : perfect vs perfect -----\n";
    std::vector<char> board(9, EMPTY);
    char player = HUMAN;                    // X 先手
    while (winnerOf(board) == EMPTY && !boardFull(board)) {
        int s = 0;
        const int m = bestMove(board, player, s);
        board[m] = player;
        player = (player == HUMAN) ? AI : HUMAN;
    }
    printBoard(board);
    const char w = winnerOf(board);
    std::cout << (w == EMPTY ? "Result: draw" : std::string("Winner: ") + w) << "\n";
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
----- Demo 1 : must block -----
  X | . | .
 ---+---+---
  . | X | .
 ---+---+---
  O | . | .
AI ('O') chooses cell 8 (row 3, col 3), evaluated score = 0
```

```
----- Demo 2 : perfect vs perfect -----
  X | X | O
 ---+---+---
  O | O | X
 ---+---+---
  X | O | X
Result: draw
```

怎么读这份输出：第一个演示摆了一个局面——X 已经在左上和正中连成一条对角线， 下一步落在右下角就赢了。AI 选择了 8 号格（第 3 行第 3 列）把路堵死，**评分为 0，意思是「只要这么走，最坏也是平局」。** 第二个演示让两个都用极小极大的对手互相下完一整局，结果是平局—— **这不是巧合，而是「双方都不犯错时，井字棋必然平局」这一数学事实，被你的程序自己证明了。**

> **这段代码美在哪里**
>
> 1. **board 按值传递，是整个设计里最漂亮的一手。** 因为传的是副本，函数里可以放心地 `board[i] = player;` 试走一步， 递归返回之后什么都不用管——**「撤销」这个动作被语言的机制免费完成了。** 如果传的是引用，你就必须记得在每一层手动撤回，多一条 return 路径就是多一个 bug。
> 2. **分数设计藏着一个精妙的心思。** `10 - depth` 和 `depth - 10`： 赢要赢得快，输要输得晚。因为「输得晚」意味着对手还得再想几步，也就意味着中间还有翻盘的机会。 **两个减号，就把「效率」这个额外目标写进了评价体系。**
> 3. **统一用一套搜索处理双方。** 同一个 `minimax` 既能当 AI 的大脑，也能当「模拟对手」的大脑， 只需要换一个 `player` 参数。**没有写两份对称的代码——这是抽象的胜利。**
> 4. **「选」与「评」分离。** `minimax` 只回答「这个局面值多少分」， `bestMove` 只回答「那我该走哪一步」。两个问题分开问，两个函数分开写， 于是 `minimax` 可以被原封不动地复用到别的地方。
> 5. **用 `numeric_limits` 而不是魔法数字。** `std::numeric_limits<int>::min()` 一眼就说明「我要一个尽可能小的起点」， 而 `-99999` 只会让读者停下来猜：为什么是九万九千九百九十九？

常见陷阱

- **判断输赢和判断下满的顺序不能反。** 必须**先看有没有人赢**，再看是不是满了。 否则最后一步落子成五的情况会被判成平局。
- 如果改成按引用传递，**每一条 `return` 之前都必须撤销试走的那一步**。 漏掉任何一条，棋盘就会带着「幽灵棋子」继续往下算。这是回溯算法最经典的 bug。
- 极小极大的代价是「分支数的深度次方」。井字棋最多 9! ≈ 36 万种局面，秒出结果； 但换成五子棋（15×15），同一段代码会把你的电脑算到天荒地老。**算法正确，不等于算法可用。**
- 递归太深会撑爆调用栈。井字棋最多 9 层，绝对安全； 但如果你把棋盘改成 20×20，程序会以「栈溢出」的方式崩溃，而不是算得慢。

进阶挑战

- **加入 alpha-beta 剪枝**：如果已经知道某条分支不可能更好，就根本不用搜它。 在井字棋上，它能把搜索量减少到十分之一左右，而结果一模一样。**这是本手册里性价比最高的一个优化。**
- 加一个计数器，统计搜索了多少个局面，对比剪枝前后的数字。你会对「优化」这个词有实感。
- 用 `map` 把「局面 → 分数」缓存下来，避免重复计算同一个局面（这就是记忆化搜索）。 你会看到搜索次数骤降，而结果一丝不差。
- 把它变成一个可以真正对弈的程序：用 `cin` 读入玩家落子，先让你自己输几局。
- 推广到四子棋（Connect Four）的 7×6 棋盘。你需要加上「只往前看 N 层」的限制和一个粗略的局面评估函数—— **恭喜，你已经摸到了现代博弈程序的核心思路。**

---

07 像数学一样的矩阵类

让 C = A * B 这四个字符，真的能跑起来

难度 ★★★☆☆ | 建议 6-8 小时 | 主线：类与运算符重载 | 阶段三

本板块会用到的新知识点

这是你第一次写自己的类型。别紧张——类不是新语言，它只是「把数据和函数放进同一个盒子里」。

- **class 与封装** 把数据（`rows_`、`cols_`、`data_`）和操作它们的函数装进同一个盒子。`private:` 的东西外面碰不到，`public:` 的东西才是你交给外界的接口。**封装的意义不是保密，而是「让使用者只需要知道他能做什么」。**
- **构造函数与初始化列表** 形如 `Matrix(int rows, int cols) : rows_(rows), cols_(cols), data_(...) {}` 保证对象一诞生就是完整可用的，而不是「先造一个空的，再慢慢填」。
- **运算符重载** 形如 `Matrix operator*(const Matrix& other) const` 让自定义类型也能用 `+` 和 `*`。**这是 C++ 最迷人的能力之一：你可以让自己的类型，长得和 int 一样自然。**
- **std::ostream& 与 << 重载** 让 `cout << A` 直接打印一个矩阵。写完这一行， 打印就从「调用一个函数」变成了「这个类型与生俱来的能力」。
- **const 成员函数** 函数声明末尾的那个 `const` 它承诺「这个函数不会改动对象本身」。有了它，`const Matrix&` 才能真正被用起来—— **这是 C++ 把「只读」这件事做进类型系统的方式。**
- **用一维数组模拟二维（行主序）** `data_[r * cols_ + c]`。 内存里是一整条连续的带子，`at()` 负责把二维坐标翻译成这条带子上的位置。 **外面的人完全不需要知道这件事——这就是封装的价值。**

一、这件事在真实世界里长什么样

你的专业方向是人工智能，而人工智能在今天几乎等同于「深度神经网络」。神经网络的一层，说穿了就是一句话： **输出向量 = 激活函数(权重矩阵 × 输入向量 + 偏置向量)**。

所以矩阵乘法就是那台机器的心脏。在 Python 里你将来会写 `A @ B`，一行搞定； 但那一行背后到底是什么？——**就是这个项目里你要亲手写出来的三重循环。**

你现在写的这个 `Matrix` 类，虽然只有一百来行，却已经具备了工业级矩阵库的骨架： 私有数据、受控访问、算术运算符、流式输出。

二、任务拆解：三关

Lv1 先用最朴素的方式实现一次

用 `std::vector<std::vector<double>>` 存一个 3×3 矩阵， 写两个普通函数：一个转置，一个打印。

这一关不用类

，你需要先感受一下「到处传二维数组」有多啰嗦——那份啰嗦，正是下一关的动力。

Lv2 **把它装进一个类**

定义 `class Matrix`：数据放 private，提供构造函数、`rows()`、`cols()`、两个版本的 `at(r,c)`，以及 `transpose()`。

检验标准：main 里再也看不到 `data_[...]` 这样的下标计算。

Lv3 **让它会做数学**

加上 `operator+`、`operator*` 和 `operator<<`。改完之后，`main` 里应该只剩下数学表达式本身——

当你能把 `A * A.transpose()` 当成一句话写出来的时候，这一关就过了。

三、参考实现

ways-to-cpp-code / p07_matrix.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 07  一个像数学一样写字的矩阵类
//  编译：  g++ -std=c++14 -Wall p07_matrix.cpp -o p07
//  运行：  ./p07
// ============================================================

#include <iomanip>
#include <iostream>
#include <vector>

// 类的意义：把"数据"和"操作这些数据的函数"捆在一起，
// 让外面的人只看到 C = A * B，而不是一堆循环。
class Matrix {
public:
    // 构造函数：对象一诞生就是合法可用的（全部填 0）
    Matrix(int rows, int cols)
        : rows_(rows), cols_(cols), data_(static_cast<size_t>(rows) * cols, 0.0) {}

    static Matrix identity(int n) {
        Matrix m(n, n);
        for (int i = 0; i < n; ++i) m.at(i, i) = 1.0;
        return m;
    }

    int rows() const { return rows_; }
    int cols() const { return cols_; }

    // 两个版本：非 const 版本可读可写，const 版本只读。
    // 编译器靠对象本身是否 const 来挑，我们什么都不用管。
    double&       at(int r, int c)       { return data_[static_cast<size_t>(r) * cols_ + c]; }
    double        at(int r, int c) const { return data_[static_cast<size_t>(r) * cols_ + c]; }
```

```
    Matrix operator+(const Matrix& other) const {
        Matrix result(rows_, cols_);
        for (int r = 0; r < rows_; ++r)
            for (int c = 0; c < cols_; ++c)
                result.at(r, c) = at(r, c) + other.at(r, c);
        return result;
    }

    Matrix operator*(const Matrix& other) const {
        Matrix result(rows_, other.cols_);
        for (int r = 0; r < rows_; ++r) {
            for (int k = 0; k < cols_; ++k) {
                const double a = at(r, k);
                for (int c = 0; c < other.cols_; ++c) {
                    result.at(r, c) += a * other.at(k, c);
                }
            }
        }
        return result;
    }

    Matrix transpose() const {
        Matrix result(cols_, rows_);
        for (int r = 0; r < rows_; ++r)
            for (int c = 0; c < cols_; ++c)
                result.at(c, r) = at(r, c);
        return result;
    }

private:
    int rows_;
    int cols_;
    std::vector<double> data_;   // 用一维数组存放，行主序
};

// 重载 << 之后，cout << A 就能像打印一个数字那样打印一个矩阵
std::ostream& operator<<(std::ostream& os, const Matrix& m) {
    for (int r = 0; r < m.rows(); ++r) {
        os << "[ ";
        for (int c = 0; c < m.cols(); ++c) {
            os << std::setw(8) << m.at(r, c) << " ";
        }
        os << "]\n";
    }
    return os;
}

int main() {
    Matrix a(3, 3);
    Matrix b(3, 3);
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            a.at(r, c) = r * 3 + c + 1;          // 1..9
            b.at(r, c) = (r == c) ? 2.0 : 0.0;   // 2 倍单位阵
```

```
        }
    }

    std::cout << std::fixed << std::setprecision(1);
    std::cout << "A =\n" << a;
    std::cout << "A + A =\n" << (a + a);
    std::cout << "A * B  (B = 2I) =\n" << (a * b);
    std::cout << "A transposed =\n" << a.transpose();
    std::cout << "A * A^T  (对称矩阵，神经网络里的协方差就是这个形状) =\n"
              << (a * a.transpose());
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
A =
[      1.0      2.0      3.0 ]
[      4.0      5.0      6.0 ]
[      7.0      8.0      9.0 ]
A + A =
[      2.0      4.0      6.0 ]
[      8.0     10.0     12.0 ]
[     14.0     16.0     18.0 ]
A * B  (B = 2I) =
[      2.0      4.0      6.0 ]
[      8.0     10.0     12.0 ]
[     14.0     16.0     18.0 ]
A transposed =
[      1.0      4.0      7.0 ]
[      2.0      5.0      8.0 ]
[      3.0      6.0      9.0 ]
A * A^T  (对称矩阵，神经网络里的协方差就是这个形状) =
[     14.0     32.0     50.0 ]
[     32.0     77.0    122.0 ]
[     50.0    122.0    194.0 ]
```

这段代码美在哪里

1. **它让代码长得像数学。** 请对比这两行：`result[r][c] += a[r][k] * b[k][c];`（在 main 里三层循环）与 `Matrix c = a * b;`。 后者不只是短，它**把「矩阵乘法」这个概念从实现细节里解放了出来。** 以后每次看到 `a * b`，你脑子里想的是数学，而不是下标。
2. **一维存储 + at() 翻译，是一次漂亮的「内外分离」。** 外面的人写 `m.at(2, 3)`，脑子里想的是「第三行第四列」； 里面的人知道那其实是第 `2 * cols_ + 3` 个元素。 **两种视角各得其所，谁也不打扰谁。**
3. **两个版本的 at() 是一个教科书级的技巧。** 非 const 版本返回 `double&`（可以赋值）， const 版本返回 `double`（只能读）。编译器根据「你手里的 Matrix 是不是 const」自动挑选， 于是只读的矩阵**在编译期就获得了不可修改的保证**——不需要运行时检查，一分钱开销都没有。
4. **构造函数保证了「不存在残缺的对象」。** 一个 Matrix 只要被造出来， 就一定有合法的行列数、一定有一块已初始化为 0 的内存。**没有「半成品状态」，就没有对应的那一类 bug。**
5. **operator<< 写成了自由函数，而不是成员函数。** 因为 `cout << A` 里， `cout` 在左边、`A` 在右边；如果写成 `A.operator<<(cout)`， 读起来就完全别扭了。**为了「读起来对」而调整代码结构的写法，值得。**

常见陷阱

- **绝对不要返回局部变量的引用。** `Matrix& operator+(...)` 返回一个函数内部创建的矩阵的引用， 是在返回一块已经销毁的内存。**这种错误编译器常常不会报，程序会以一种诡异的方式在别处崩溃。** 算术运算符**一律返回值**。
- 忘了写 const 成员函数，会导致 `const Matrix&` 参数上调用 `.at()` 时编译不过。 错误信息会很长很吓人，但原因就是这一个词。
- 矩阵乘法要求 `A.cols() == B.rows()`。参考实现为了短小没有检查， 真实代码里应该在函数开头就挡住，并且**明确地报错**。
- 这里的数据成员用了 `std::vector`，所以默认的拷贝行为是正确的（值语义）。 如果你改用 `new double[...]` 裸指针，就必须自己写析构函数和拷贝构造函数—— **这就是「用标准库容器」帮你省掉的一大类麻烦。**

进阶挑战

- 加上 `operator-`、`operator==` 和「矩阵 × 标量」。 注意标量乘法要写成自由函数 `operator*(double k, const Matrix& m)`， 否则 `2.0 * A` 这种写法就编译不过——**想想为什么。**
- 实现 `identity(n)`（已经是静态成员函数了）、行列式、以及高斯消元求逆。
- **用这个类写一个两层全连接网络的前向传播：**`h = relu(W1 * x + b1); y = W2 * h + b2;`。 你会突然发现，深度学习框架里那些抽象的 API，在你手里变成了看得见摸得着的几行代码。**这是本项目最有价值的一步。**
- 把 `Matrix` 拆成 `matrix.h` 和 `matrix.cpp` 两个文件， 体验一次真正的「接口与实现分离」。头文件开头记得加 `#pragma once`。

08 训练你的第一个模型：感知机

五十行代码，一个完整的机器学习闭环

难度 ★★★★☆ | 建议 6-9 小时 | 主线：学习即修正 | 阶段三

本板块会用到的新知识点

这个项目的新知识里，只有一个是纯语法的（随机数），其余都是「方法」——而方法才是你专业里真正值钱的东西。

- **<random> 随机数引擎** `std::mt19937 engine(种子);` 和 `std::normal_distribution<double> dist(0.0, 0.8);` 别再使用 `rand()` 了，它质量差且分布不均。以及一个关键技巧：**给定固定种子，随机数序列就是可复现的**—— 这是所有实验科学的底线。
- **「模型即参数」的世界观** 这个感知机类的全部内容就是三个 `double`： `w1_`、`w2_`、`b_`。**所谓「训练」，就是不断调整这三个数字； 所谓「模型文件」，就是把这三个数字存下来。**复杂的深度学习，本质结构也是这样，只是数字从三个变成了几十亿个。
- **训练循环** 「预测 → 和正确答案比较 → 错了就朝正确方向挪一小步」 重复很多很多次。这一页纸就能写完的循环，是整个人工智能的地基。
- **参数持久化** 把训练好的参数写进文件、下次读回来。 **程序第一次拥有了「记忆」**，也第一次体现出了「训练一次、使用很多次」这个真实的工程流程。
- **std::setprecision 与有效数字** 需要 `#include <iomanip>` 存文件时用 `setprecision(17)` 保证 double 不丢精度，屏幕上显示时用 3 位就够。 **同一个数字，写给机器看和写给人看，要求完全不同。**

一、这件事在真实世界里长什么样

一九五七年，弗兰克·罗森布拉特造出了感知机。它要做的事情朴素得可爱： **在平面上画一条直线，把两类点分开。**

它学习的方式更可爱：随便画一条线，然后一个点一个点地看过去——分对了就不动，分错了就把线朝那个点的方向掰一点。 重复几轮之后，线就自己找到了位置。

《纽约时报》当年报道说，这台机器将来能走路、能说话、能自我复制。 这个预言夸张了，但它指的方向是对的——**七十年后，你所在专业的一切，都还在这条「错一点、改一点」的路上。**

所以这个项目不是为了让你掌握感知机（它太简单了，实际没什么用）， 而是让你**亲手走完一次「造数据 → 训练 → 评估 → 保存 → 再加载使用」的完整闭环。** 这个流程的形状，和你将来用 PyTorch 训练一个大模型的流程，是一模一样的。

二、任务拆解：三关

Lv1 先让一条线会做判断

定义 `struct Sample { double x1, x2; int label; };`， 写一个函数 `predict(x1, x2)`：算 `w1*x1 + w2*x2 + b`， 大于等于 0 就返回 `+1`，否则 `-1`。

先随便给 w1、w2、b 一些值，看看这条线把空间分成了什么样。

Lv2 让它自己学

造两组数据（一组在右上、一组在左下），然后写训练循环： 对每个样本预测一次，

如果错了，就按 `w += 学习率 × 正确答案 × 输入` 更新

。 跑完一轮，统计错了几次，再跑下一轮。

看着 mistakes 从几十降到 0，是这个项目最爽的一刻。

Lv3 让它成为一个「能交付的东西」

用 `mt19937` 生成 80 个带噪声的点（固定种子）； 训练完之后算出准确率；把参数存进 `perceptron.txt`； 再新建一个对象、把参数读回来、验证它的预测和原来完全一致。

这一步做完，你的程序就不再是一个练习，而是一个可以反复使用的工具。

三、参考实现

ways-to-cpp-code / p08_perceptron.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 08  训练你的第一个模型：感知机
//  编译：  g++ -std=c++14 -Wall p08_perceptron.cpp -o p08
//  运行：  ./p08
//  说明：  训练完会把参数存进 perceptron.txt，再读回来验证一次。
// ============================================================

#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <random>
#include <string>
#include <vector>

struct Sample {
    double x1;
    double x2;
    int    label;      // +1 或 -1
};

// ---------- 模型本身：三个数字就是它的全部 ----------
class Perceptron {
```

```
public:
    Perceptron(int epochs, double learningRate)
        : w1_(0.0), w2_(0.0), b_(0.0),
          epochs_(epochs), learningRate_(learningRate) {}

    // 一次前向计算：把点投影到一个方向上，看它落在哪一边
    int predict(double x1, double x2) const {
        return (w1_ * x1 + w2_ * x2 + b_ >= 0.0) ? 1 : -1;
    }

    // 训练：错一次，就朝着"少错一点"的方向挪一小步
    void train(const std::vector<Sample>& data) {
        for (int epoch = 1; epoch <= epochs_; ++epoch) {
            int mistakes = 0;
            for (const Sample& s : data) {
                const int guess = predict(s.x1, s.x2);
                if (guess == s.label) continue;

                ++mistakes;
                w1_ += learningRate_ * s.label * s.x1;
                w2_ += learningRate_ * s.label * s.x2;
                b_  += learningRate_ * s.label;
            }
            if (epoch <= 5 || mistakes == 0) {
                std::cout << "  epoch " << std::setw(3) << epoch
                          << "   mistakes = " << mistakes << "\n";
            }
            if (mistakes == 0) break;      // 全对了，提前收工
        }
    }

    double accuracy(const std::vector<Sample>& data) const {
        if (data.empty()) return 0.0;
        int correct = 0;
        for (const Sample& s : data) {
            if (predict(s.x1, s.x2) == s.label) ++correct;
        }
        return 100.0 * correct / static_cast<double>(data.size());
    }

    void print() const {
        std::cout << std::fixed << std::setprecision(3);
        std::cout << "  w1 = " << w1_ << "   w2 = " << w2_ << "   b = " << b_ << "\n";
        std::cout << std::setprecision(4)
                  << "  boundary: " << w1_ << "*x1 "
                  << (w2_ >= 0.0 ? "+ " : "- ") << std::fabs(w2_) << "*x2 "
                  << (b_  >= 0.0 ? "+ " : "- ") << std::fabs(b_)  << " = 0\n"
                  << std::setprecision(3);
    }

    void save(const std::string& path) const {
        std::ofstream fout(path);
        fout << std::setprecision(17) << w1_ << " " << w2_ << " " << b_ << "\n";
    }
```

```
    bool load(const std::string& path) {
        std::ifstream fin(path);
        if (!fin) return false;
        fin >> w1_ >> w2_ >> b_;
        return true;
    }

private:
    double w1_, w2_, b_;
    int    epochs_;
    double learningRate_;
};

int main() {
    // ---------- 1. 造一批线性可分的数据：两团点 ----------
    std::mt19937 engine(20180101);                    // 固定种子 -> 每次运行结果一致
    std::normal_distribution<double> spread(0.0, 0.8);

    std::vector<Sample> trainSet;
    for (int i = 0; i < 40; ++i) {
        Sample s;
        s.x1 = 2.0 + spread(engine);
        s.x2 = 2.0 + spread(engine);
        s.label = 1;
        trainSet.push_back(s);
    }
    for (int i = 0; i < 40; ++i) {
        Sample s;
        s.x1 = -2.0 + spread(engine);
        s.x2 = -2.0 + spread(engine);
        s.label = -1;
        trainSet.push_back(s);
    }

    std::cout << "----- Training -----\n";
    std::cout << "  samples = " << trainSet.size() << "\n";

    Perceptron model(200, 0.01);
    model.train(trainSet);

    std::cout << "\n----- Learned parameters -----\n";
    model.print();
    std::cout << "  Training accuracy = " << model.accuracy(trainSet) << "%\n";

    // ---------- 2. 存盘，再读回来，确认模型能"隔天继续用" ----------
    const std::string path = "perceptron.txt";
    model.save(path);
    std::cout << "\n----- Persistence -----\n";
    std::cout << "  saved to " << path << "\n";

    Perceptron reloaded(200, 0.01);
    if (reloaded.load(path)) {
        std::cout << "  reloaded accuracy = " << reloaded.accuracy(trainSet) << "%\n";
```

```
    }

    // ---------- 3. 拿几个新点试试 ----------
    std::cout << "\n----- Predictions -----\n";
    const double probes[][2] = {{3.0, 3.0}, {-3.0, -3.0}, {3.0, -0.5}, {-0.5, -3.0}};
    for (int i = 0; i < 4; ++i) {
        std::cout << "  (" << std::setw(5) << probes[i][0] << ","
                  << std::setw(5) << probes[i][1] << ")"
                  << "  ->  label " << reloaded.predict(probes[i][0], probes[i][1]) << "\n";
    }
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
----- Training -----
  samples = 80
  epoch   1   mistakes = 1
  epoch   2   mistakes = 0

----- Learned parameters -----
  w1 = 0.021   w2 = 0.014   b = -0.010
  boundary: 0.0215*x1 + 0.0135*x2 - 0.0100 = 0
  Training accuracy = 100.000%

----- Persistence -----
  saved to perceptron.txt
  reloaded accuracy = 100.000%

----- Predictions -----
  (3.000,3.000)  ->  label 1
  (-3.000,-3.000)  ->  label -1
  (3.000,-0.500)  ->  label 1
  (-0.500,-3.000)  ->  label -1
```

怎么读这份输出：第一轮错了 1 个点，第二轮就一个都不错了。 最后得到的分界线是 `0.0215*x1 + 0.0135*x2 - 0.0100 = 0`， 也就是说这条线大致是 `x1 + 0.63*x2 ≈ 0.47` —— **它不是你想当然的 `x1 + x2 = 0`，而是数据自己选出来的一个倾斜角度。** 最后四个测试点里，`(3.0, -0.5)` 落在右下角却仍被判为 `+1`， 正好说明这条边界是被数据「掰」过的，而不是凭直觉划的。

这段代码美在哪里

1. **整个模型的全部内容，就是三个 double。** 没有复杂的结构，没有玄妙的技巧。 **越是强大的想法，它的核心往往越是简单**——感知机的核心就是「错了就往正确方向挪一点」。 理解这一点，你对「机器学习」就不会再有神秘感。
2. **三行更新规则长得一模一样。** `w1_ += learningRate_ * s.label * s.x1;` 、 `w2_ += ... * s.x2;` 、 `b_ += ... * 1;` 。 最后那个 `× 1` 是暗含的。这种规律性不是巧合，它暗示着「如果有 1000 个特征，你就需要一个循环」——**而那正是从感知机走向真正神经网络的那一步。**
3. **predict() 是 const 成员函数，train() 不是。** 一个只读、一个会改，签名上分得清清楚楚。**光看函数名和 const，你就能知道哪个函数会改变世界。**
4. **固定随机种子是一种科学态度。** `std::mt19937 engine(20180101);` —— 有了它，你今天的实验结果和明天的完全一致。做研究的人都知道， **不能复现的实验等于没做。**
5. **save / load 成对出现，而且 load 会告诉你成功还是失败。** `bool load(...)` 返回布尔值，调用方可以据此决定是继续还是报错。 **「错误是返回值，不是沉默」——这是工程代码的分水岭。**
6. **输出精度分了两档。** 存文件用 `setprecision(17)` ， 屏幕显示用 3 位小数。**同一个数据，面对机器和面对人，应该有不同的呈现方式。**

常见陷阱

- **学习率是个需要手感的东西。** 太大（比如 10）会让参数来回震荡、永远不收敛； 太小（比如 0.000001）则跑到天荒地老。0.01 到 0.1 之间通常是个好起点。 **「调参」这个词，你在这里第一次体会到它的真实含义。**
- 一定要给训练轮数设上限。如果数据本身不可分（两团点混在一起），感知机永远不会停下—— 这是感知机算法本身的局限，不是 bug。
- `rand()` 生成的整数虽然方便，但周期短、低位随机性差。 既然标准库已经给了 `mt19937` ，就没有理由再将就。
- 存文件时如果只写 `setprecision(6)` ，读回来的参数会有微小误差， 导致重新加载后准确率变化。**浮点数写进文本文件时，默认精度是不够的。**

进阶挑战

- **把「错了才改」改成「每次都按误差大小改」：**`w += lr * (label - prediction) * x`， 注意此时 prediction 要用输出值而不是符号。这就是**梯度下降**，比感知机更通用的那一套。
- 把二维扩展到三维、十维。你会发现代码几乎不用改，只需要把 `w1_, w2_` 换成 `std::vector<double> weights_`，并把点积写成一个循环。 **恭喜，你刚刚完成了从「感知机」到「线性分类器」的抽象。**
- 把数据换成真实数据：用项目 02 的文件读取方法，从 `data.txt` 读入点的坐标和标签。
- 每轮训练后把分界线画在网格上（用项目 03 的二维网格），**你会亲眼看到那条线是怎么一步步转过去的。**

09 朴素贝叶斯垃圾信息分类器

用你亲手写的代码，读一封邮件是不是垃圾

难度 ★★★★☆ | 建议 6-9 小时 | 主线：从数据里学概率 | 阶段三

本板块会用到的新知识点

这个项目的代码不长，但有两处「第一次遇到一定会踩」的数值陷阱。它们也是本手册最重要的新知识。

- **把概率取对数** 一句话概率是若干个小概率相乘，几十个词乘下来，结果小到 `double` 都表示不了， 直接变成 0（这叫下溢）。**解法：取对数，乘法变加法。**因为对数函数是单调递增的，比大小不受影响，而加法永远不会下溢。**这是数值计算里的必修课。**
- **拉普拉斯平滑** 如果一个从没见过的词出现，它的概率算出来是 0，那么整个连乘结果立刻变成 0， 整条判断就此作废。**解法：分子加 1、分母加上词表大小**，给每个词留一条活路。 这一行小小的修正，是朴素贝叶斯能真正工作的前提。
- `std::log` 需要 `#include <cmath>` 自然对数。本项目中它的唯一用途就是「把乘法变加法」。
- **map 的 find 与 const 查询** `table_.find(w)` 返回一个迭代器，找不到就是 `end()` 项目 02 里我们用了 `counter[w]`，但那会在表里插一个新条目。 在**只读的预测函数里**，我们不能再改表，所以必须用 `find`。
- **map 套结构体** `std::map<std::string, WordStats>`。 把「这个词在垃圾里出现几次、在正常里出现几次」打包成一个结构体， 于是**两张表合并成了一句话可以说清的一张表。**
- **训练与预测的彻底分离** 训练时更新统计量，预测时**只读**。 这意味着训练完的程序可以只保存那张统计表，运行时根本不需要原始语料。**这就是「模型」与「数据」的分家。**

一、这件事在真实世界里长什么样

你每天收到的垃圾邮件，绝大多数是被这类程序拦下来的。它们的思路非常人类：

「这封邮件里有『免费』『中奖』『点击』这些词。我在以前看过的垃圾邮件里，这些词出现得特别多； 在正常邮件里几乎不出现。所以这封大概也是垃圾。」

把这句直觉翻译成数学，就是贝叶斯定理。而之所以叫「朴素」，是因为它做了一个明显不成立的假设： **假设所有词之间互相独立**——「免」和「费」显然不是独立的。

但神奇的是，这个「错」的假设，在实际任务里效果出奇地好，而且快得不可思议。 **「理论上有瑕疵，工程上却好用」，这种取舍在人工智能里随处可见，你要早点习惯它。**

二、任务拆解：三关

Lv1 先学会「数案底」

用项目 02 的切词函数，统计语料里每个词在垃圾消息和正常消息里各出现了几次， 顺便记下两边各有多少个词、各有多少条消息。

这一关只做统计，不做任何判断。

Lv2 **从次数算出概率**

算 `P(这个词 | 这是垃圾)`：

分子是它出现过的次数加一，分母是垃圾消息的总词数加词表大小

。 那个「加一」和「加词表大小」就是拉普拉斯平滑。

亲手去掉平滑那一项，看看程序会怎样崩掉——这个实验比记住规则有用得多。

Lv3 **取对数再比大小**

对一条新消息：把所有词的 `log P(词|垃圾)` 加起来，再加上 `log P(垃圾)`； 对正常消息做同样的计算。哪个大就算哪一类。

注意分数都是负数，比的是「谁更接近 0」。

三、参考实现

ways-to-cpp-code / p09_naive_bayes.cpp

```cpp
// ============================================================
// Ways to cpp · 项目 09 朴素贝叶斯垃圾信息分类器
// 编译: g++ -std=c++14 -Wall p09_naive_bayes.cpp -o p09
// 运行: ./p09
// ============================================================

#include <cctype>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <vector>

// 一个词在两边的"案底"
struct WordStats {
    int spam;
    int ham;
    WordStats() : spam(0), ham(0) {}
    int total() const { return spam + ham; }
};

typedef std::map<std::string, WordStats> Table;

// 把一句话切成小写单词（和项目 02 是同一件事，直接拿来用）
std::vector<std::string> splitWords(const std::string& text) {
    std::vector<std::string> words;
    std::string current;
```

```
    for (char raw : text) {
        const unsigned char ch = static_cast<unsigned char>(raw);
        if (std::isalpha(ch)) {
            current += static_cast<char>(std::tolower(ch));
        } else if (!current.empty()) {
            words.push_back(current);
            current.clear();
        }
    }
    if (!current.empty()) words.push_back(current);
    return words;
}

// ---------- 分类器 ----------
class NaiveBayes {
public:
    NaiveBayes() : spamDocs_(0), hamDocs_(0), spamWords_(0), hamWords_(0) {}

    void train(const std::string& text, bool isSpam) {
        const std::vector<std::string> words = splitWords(text);
        if (isSpam) ++spamDocs_; else ++hamDocs_;

        for (const std::string& w : words) {
            WordStats& s = table_[w];          // 没见过的词会自动建一条空记录
            if (isSpam) { ++s.spam; ++spamWords_; }
            else        { ++s.ham;  ++hamWords_;  }
        }
    }

    // 返回对数概率：log P(垃圾) + Σ log P(词 | 垃圾)
    double spamScore(const std::string& text) const {
        const double prior = std::log(static_cast<double>(spamDocs_) /
                                      (spamDocs_ + hamDocs_));
        return prior + logLikelihood(text, true);
    }

    double hamScore(const std::string& text) const {
        const double prior = std::log(static_cast<double>(hamDocs_) /
                                      (spamDocs_ + hamDocs_));
        return prior + logLikelihood(text, false);
    }

    // 预测：哪边的分数大就归哪边
    bool isSpam(const std::string& text) const {
        return spamScore(text) > hamScore(text);
    }

    size_t vocabularySize() const { return table_.size(); }

private:
    // 拉普拉斯平滑：分子 +1，分母 + 词表大小，
    // 这样"从没见过的词"也不会把概率算成 0（一旦是 0，取对数就炸了）
    double logLikelihood(const std::string& text, bool spam) const {
        const std::vector<std::string> words = splitWords(text);
```

```
        const double vocab = static_cast<double>(table_.size());
        const double denom = (spam ? spamWords_ : hamWords_) + vocab;
        double sum = 0.0;

        for (const std::string& w : words) {
            const Table::const_iterator it = table_.find(w);
            const int count = (it == table_.end()) ? 0 : (spam ? it->second.spam
                                                               : it->second.ham);
            sum += std::log((count + 1.0) / denom);
        }
        return sum;
    }

    Table table_;
    int spamDocs_;
    int hamDocs_;
    int spamWords_;
    int hamWords_;
};

int main() {
    // ---------- 训练集：12 条短消息 ----------
    struct Row { const char* text; bool spam; };
    const Row rows[] = {
        {"win a free iphone now click the link",              true},
        {"cheap loan approved instantly no credit check",     true},
        {"congratulations you won a lottery prize claim",     true},
        {"buy discount pills online free shipping now",       true},
        {"urgent your account will be closed click here",     true},
        {"free entry to win cash prize limited offer",        true},
        {"meeting moved to three pm in room b",               false},
        {"please review the attached report before friday",   false},
        {"lunch today at the campus canteen",                 false},
        {"the library book is due next week",                 false},
        {"can you send me the notes from lecture",            false},
        {"the exam schedule is posted on the website",        false}
    };

    NaiveBayes model;
    for (int i = 0; i < 12; ++i) {
        model.train(rows[i].text, rows[i].spam);
    }
    std::cout << "Trained on 12 messages, vocabulary = "
              << model.vocabularySize() << " words\n\n";

    // ---------- 测试 ----------
    const std::string tests[] = {
        "free cash prize click now",
        "can we meet after the lecture",
        "your loan is approved with no credit check",
        "the report is attached for review"
    };

    std::cout << std::fixed << std::setprecision(2);
```

```
    std::cout << "----- Predictions -----\n";
    for (const std::string& t : tests) {
        const double s = model.spamScore(t);
        const double h = model.hamScore(t);
        std::cout << std::left << std::setw(44) << t
                  << " spam=" << std::setw(8) << s
                  << " ham="  << std::setw(8) << h
                  << "  ->  " << (model.isSpam(t) ? "SPAM" : "ham") << "\n";
    }
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
Trained on 12 messages, vocabulary = 73 words

----- Predictions -----
free cash prize click now                    spam=-19.17   ham=-24.50    ->  SPAM
can we meet after the lecture                spam=-28.62   ham=-25.93    ->  ham
your loan is approved with no credit check   spam=-34.70   ham=-37.69    ->  SPAM
the report is attached for review            spam=-28.62   ham=-24.14    ->  ham
```

怎么读这份输出：训练集只有十二条消息，而四条测试全部判对了——包括 `"your loan is approved with no credit check"` 这种在训练集里从未完整出现过的句子。分数都是负数，这是正常的：四五个小概率相乘，取对数之后就是若干个负数的和。**比的不是谁大谁小，而是谁更接近 0。**

> **这段代码美在哪里**
>
> 1. **一个「错」的假设，换来了极致的简单与速度。**朴素贝叶斯假设词与词互不相关，这在语言上显然是错的；但正因为这个假设，概率才能拆成一个个独立的乘积，计算量从「所有词的组合」降到「一次遍历」。**教科书告诉你什么是对的，工程告诉你什么是有用的。**
> 2. **取对数这一步，是「换个角度看问题」的典范。**概率连乘和 log 概率连加，在数学上完全等价，在计算机上却是「算不出来」和「轻松算完」的差别。**算法工程师的功夫，一半花在这种转换上。**
> 3. **WordStats 结构体把两张表合成了一张。**如果写成两个 `map`，每次查询都要写两遍、还要保证两边同步。一个五行的小结构体，让「这个词的两边案底」变成了一个不可分割的整体。**数据该怎么分组，本身就是一种设计。**
> 4. **训练改数据，预测只读数据，靠 const 强制分开。**`spamScore(...) const` 里的 const 不是装饰，它保证了「预测这件事绝对不会污染模型」——**而这正是「推理」和「训练」必须分开的原因。**
> 5. **每个数字都可以解释。**随便挑一条预测，你都能手工算出那个 -19.17 是怎么来的。在深度学习时代，这种「可解释性」变得非常稀有——**所以趁现在，先体会一次「我知道它为什么这么判断」的踏实感。**

常见陷阱

- **不取对数一定会在真实数据上崩溃。** 用十二条短消息可能侥幸不溢出， 但换成一千封真实邮件，几百个小概率相乘会稳稳地变成 0，然后两边都是 0，程序彻底失效。
- **不平滑同样会崩溃。** 只要测试句子里有一个训练集没见过的词，它的概率就是 0， 一乘之下全盘归零。**你要能在不看答案的情况下说出：为什么「加一」能解决这个问题。**
- 别忘记加上先验概率 `log P(垃圾)`。如果训练集里正常邮件是垃圾邮件的十倍， 那么一条中性消息应该更倾向于被判为正常——这个信息就藏在先验里。
- 这套方法对中文几乎无效，因为它依赖空格切词。 **中文需要先做分词，而分词本身就是一个不小的课题**——这也是你将来在 Python 里会遇到的第一个真实障碍。
- 训练集太小的时候，准确率高不代表模型好。 十二条消息上 100% 的准确率，没有任何统计意义。**这是你对「过拟合」最早的直觉。**

进阶挑战

- 把训练语料换成文件（每行一条，前面用 `spam:` 或 `ham:` 标注）， 扩大到一百条以上，观察准确率的变化。
- 加一个「停用词表」，把 `the`、`a`、`is` 这类词排除。 **你会看到一个反直觉的结果：有些看似无用的词，其实对分类有帮助。**
- 输出每个词对判断的「贡献值」`log P(w|spam) - log P(w|ham)`，按绝对值排序。 这份榜单就是这个模型的**全部知识**——它把自己学到的东西，用人能读懂的方式告诉了你。
- 把「词」升级成「词对」（项目 02 的进阶挑战做过）。准确率会上升，泛化能力会下降， **这就是「模型复杂度」的两难，你在这里第一次亲手摸到它。**
- 加一个混淆矩阵：把测试结果分成「判对/判错 × 垃圾/正常」四类。 你会发现「把正常邮件误判为垃圾」的代价，远比反过来高——**这就是「损失函数」这个概念的现实来源。**

---

10 曼德博集合与图像

六行数学，一张无限复杂的图

难度 ★★★☆☆ | 建议 5-8 小时 | 主线：把程序变成产品 | 阶段四

本板块会用到的新知识点

最后一个项目。你将第一次写一个「输出不是文字」的程序——而这个差别，比它看起来重要得多。

- **二进制文件输出** `std::ofstream out(path, std::ios::binary);` 配合 `out.put(字节)` 以前写的是给人看的文本，这次写的是给程序看的字节。**一旦掌握了这一招，你的程序就能生成图片、音频、模型文件——它开始产出「作品」而不只是「结果」。**
- **BMP 文件格式** 一个非常老、非常直白的图片格式：54 字节文件头 + 调色板 + 像素数据。它是最好的「文件不过是一串字节」的教材。
- **小端序与字节拆装** 形如 `out.put(value & 0xFF); value >>= 8;` 一个 int 是 4 个字节，写进文件时需要决定谁先谁后。BMP 规定低位字节在前。
- **坐标映射** `cr = -2.20 + 3.00 * x / WIDTH` 把「第 x 列像素」翻译成「复平面上的某个数」。**可视化的第一步永远是这件事：把屏幕坐标翻译成数据坐标。**将来你画损失曲线、画决策边界，用的都是同一个映射。
- **一维缓冲区当二维用** `gray[y * WIDTH + x]` 内存在物理上永远是一条线。项目 07 的矩阵内部也是这么存的，这里你自己动手算一次下标，感受会更实在。

一、这件事在真实世界里长什么样

取一个复数 `c`，令 `z` 从 0 开始，反复做同一件小事：$z = z^2 + c$。

有的 `c` 会让 `z` 越跑越远，有的会让它老老实实待在一个小圈子里。把「待得住」的那些 `c` 涂黑，就得到了曼德博集合。

规则就这么一句话。可当你把它画出来——你会看到一个边缘无限复杂的图形：**无论放大多少倍，新的细节都会继续冒出来，而且每一处都隐约带着整个图形的影子。**

这个项目想让你体会的是：**程序的产出，可以不只是屏幕上的几行数字。**你的代码第一次会生成一个文件，这个文件可以被别的软件打开、可以发给别人、可以印在纸上。**这就是「产品」和「作业」的区别。**

二、任务拆解：三关

Lv1

先算一个点

写一个函数：给定 `c`，迭代 $z = z^2 + c$ 最多 200 次，每次检查 $|z|^2 > 4$ 就提前返回「第几次跑掉的」。

先在 main 里手工试几个 c，比如 (0,0)、(1,1)、(-1,0)，看它们的逃逸次数差多少。

Lv2 **把整张画面算出来**

开一个 480×320 的 `vector<unsigned char>`，对每个像素：先把行列号映射成复平面的坐标，算出逃逸时间，再映射成 0–255 的灰度，存进缓冲区。

在终端用字符画先看一眼——只要形状对了，后面就都好办。

Lv3 **写成一张真正的图片**

按 BMP 的格式写文件：14 字节文件头 + 40 字节信息头 + 256 级灰度调色板 + 像素数据。

三个必须记住的细节：每行字节数补齐到 4 的倍数、像素自下而上存储、写完用图片查看器打开它。

三、参考实现

ways-to-cpp-code / p10_mandelbrot.cpp

```cpp
// ============================================================
//  Ways to cpp · 项目 10  曼德博集合：把数学画成一张图
//  编译：  g++ -std=c++14 -Wall p10_mandelbrot.cpp -o p10
//  运行：  ./p10
//  产物：  mandelbrot.bmp（双击就能看）
// ============================================================

#include <algorithm>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

const int WIDTH  = 480;
const int HEIGHT = 320;
const int MAX_ITER = 200;

// 复数 z 的平方加 c。C++ 标准库里有 <complex>，
// 但这里手写四行，反而更清楚每一次乘法去了哪里。
void squareAndAdd(double& zr, double& zi, double cr, double ci) {
    const double newR = zr * zr - zi * zi + cr;
    const double newI = 2.0 * zr * zi + ci;
    zr = newR;
    zi = newI;
}

// 点 (cr, ci) 逃逸得越慢，说明它越"靠近"集合，颜色越黑。
// 返回值 0..255，0 表示被判定为在集合内。
int escapeTime(double cr, double ci) {
    double zr = 0.0, zi = 0.0;
```

```
    for (int i = 0; i < MAX_ITER; ++i) {
        squareAndAdd(zr, zi, cr, ci);
        if (zr * zr + zi * zi > 4.0) {          // 跑出半径 2 的圆，就再也回不来了
            return 255 - (i * 255) / MAX_ITER;  // 跑得越快越白，慢的越黑
        }
    }
    return 0;
}

// ---------- 把灰度缓冲区写成一张 BMP 图片 ----------
// BMP 头是"老式但极其直白"的二进制格式，正好用来学"文件不过是字节"。
void writeInt(std::ofstream& out, int value) {
    for (int i = 0; i < 4; ++i) {               // 小端序：低位字节在前
        out.put(static_cast<char>(value & 0xFF));
        value >>= 8;
    }
}

void writeShort(std::ofstream& out, int value) {
    out.put(static_cast<char>(value & 0xFF));
    out.put(static_cast<char>((value >> 8) & 0xFF));
}

bool saveBmp(const std::string& path, int w, int h, const std::vector<unsigned char>& gray) {
    const int rowBytes = (w + 3) / 4 * 4;       // BMP 规定每行字节数必须是 4 的倍数
    const int dataSize = rowBytes * h;
    const int fileSize = 54 + 1024 + dataSize;  // 头 54 字节 + 256 级灰度调色板

    std::ofstream out(path.c_str(), std::ios::binary);
    if (!out) return false;

    out.put('B'); out.put('M');                 // 文件头
    writeInt(out, fileSize);
    writeInt(out, 0);
    writeInt(out, 54 + 1024);

    writeInt(out, 40);                          // 信息头
    writeInt(out, w);
    writeInt(out, h);
    writeShort(out, 1);
    writeShort(out, 8);                         // 每像素 8 位 = 灰度图
    writeInt(out, 0);
    writeInt(out, dataSize);
    writeInt(out, 2835);
    writeInt(out, 2835);
    writeInt(out, 256);
    writeInt(out, 256);

    for (int i = 0; i < 256; ++i) {             // 调色板：第 i 号颜色就是灰度 i
        out.put(static_cast<char>(i));
        out.put(static_cast<char>(i));
        out.put(static_cast<char>(i));
        out.put(static_cast<char>(0));
    }
```

```
    for (int y = h - 1; y >= 0; --y) {           // BMP 的像素是自下而上存的
        for (int x = 0; x < w; ++x) {
            out.put(static_cast<char>(gray[y * w + x]));
        }
        for (int p = w; p < rowBytes; ++p) out.put(static_cast<char>(0));
    }
    return true;
}

// ---------- 在终端里先看一眼：把图缩小成字符画 ----------
void printPreview(const std::vector<unsigned char>& gray, int w, int h) {
    const int stepX = 12;
    const int stepY = 16;
    const std::string ramp = "@%#*+=-:. ";       // 从黑到白
    for (int y = 0; y < h; y += stepY) {
        std::cout << "  ";
        for (int x = 0; x < w; x += stepX) {
            const int level = gray[y * w + x] * (static_cast<int>(ramp.size()) - 1) / 255;
            std::cout << ramp[level];
        }
        std::cout << "\n";
    }
}

int main() {
    std::vector<unsigned char> gray(static_cast<size_t>(WIDTH) * HEIGHT, 0);

    std::cout << "Rendering " << WIDTH << "x" << HEIGHT << " ...\n";
    for (int y = 0; y < HEIGHT; ++y) {
        const double ci = -1.15 + 2.30 * y / HEIGHT;      // 把像素坐标映射到复平面
        for (int x = 0; x < WIDTH; ++x) {
            const double cr = -2.20 + 3.00 * x / WIDTH;
            gray[static_cast<size_t>(y) * WIDTH + x] =
                static_cast<unsigned char>(escapeTime(cr, ci));
        }
    }

    const std::string path = "mandelbrot.bmp";
    if (!saveBmp(path, WIDTH, HEIGHT, gray)) {
        std::cerr << "Cannot write " << path << "\n";
        return 1;
    }
    std::cout << "Saved " << path << " (" << (54 + 1024 + WIDTH * HEIGHT) << " bytes)\n\n";
    std::cout << "----- ASCII preview -----\n";
    printPreview(gray, WIDTH, HEIGHT);
    return 0;
}
```

实际运行结果（真实输出，未经修饰）

```
Rendering 480x320 ...
Saved mandelbrot.bmp (154678 bytes)

----- ASCII preview -----
          ..............................
         ..............................
        ......................:.......
       ......................@@:.......
       ......................@@.......
      ..................-@@@@@@@@@@.@:.....
      .................=@@@@@@@@@@@@@.....
      ................-@@@@@@@@@@@@@@=....
     ...........@@@@..@@@@@@@@@@@@@@@.....
     ..........=@@@@@-@@@@@@@@@@@@@@@.....
    @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@.......
     ..........=@@@@@-@@@@@@@@@@@@@@@.....
     ...........@@@@..@@@@@@@@@@@@@@@.....
      ................-@@@@@@@@@@@@@@=....
      .................=@@@@@@@@@@@@@.....
      ..................-@@@@@@@@@@.@:.....
       ......................@@.......
       ......................@@:.......
        ......................:.......
         ..............................
```

怎么读这份输出：程序在终端里用字符先「预览」了一遍——`@` 是判定为在集合内的点（黑色），越往 `.` 方向越白。 中间那一大块就是曼德博集合的主体（心形），左边那条横线是实轴上的区间，上下对称。同时它在你运行程序的目录里生成了 `mandelbrot.bmp`，**双击就能看到完整分辨率的图像。**

BMP 之外更短的写法（PGM 格式）

如果只是想快速看到灰度图，PGM 格式只要六行：

```
// PGM 版本：比 BMP 短得多，但 Windows 默认打不开
std::ofstream out("mandelbrot.pgm", std::ios::binary);
out << "P5\n" << WIDTH << " " << HEIGHT << "\n255\n";
for (int y = 0; y < HEIGHT; ++y) {
    for (int x = 0; x < WIDTH; ++x) {
        out.put(static_cast<char>(gray[y * WIDTH + x]));
    }
}
```

代价是 Windows 的图片查看器默认不认识 `.pgm`。 用 BMP 换来的那三十行，买到的正是「双击就能看」这个体验。 这一点在真实产品里很关键：用户的方便，值得你多写代码。

这段代码美在哪里

1. 极致的投入产出比。 整个图形的数学核心只有两行： `zr = zr*zr - zi*zi + cr;` 和 `zi = 2*zr*zi + ci;`。 用最少的数学，生产出无限复杂的结构——这是数学之美，被你的代码原样搬运到了屏幕上。
2. squareAndAdd 这个函数，把「复数平方加 c」变成了一个动词。 它内部明明有四行运算，但调用处只写 `squareAndAdd(zr, zi, cr, ci);`， 读起来就是它名字说的那件事。给一段操作起一个准确的名字，是抽象的开始。
3. 整数除法的顺序被刻意安排过。 `(i * 255) / MAX_ITER` —— 先乘后除，是为了不让整数除法过早地把结果截断成 0。 这种细节看起来很小，但它决定了图像是平滑过渡还是一团死黑。
4. 文件写入被拆成了 writeInt / writeShort 两个小函数。 BMP 头有十几个字段要写，如果不拆，主函数里会出现十几行一模一样的 `out.put(...)`。 拆完之后，文件头的结构一眼就能对上 BMP 的文档。
5. 同一个缓冲区，喂给了两个完全不同的输出。 `saveBmp()` 把它变成图片，`printPreview()` 把它变成字符画。 「计算一次、呈现多次」——把数据的产生和数据的展示分开，是一个值得记住的架构习惯。
6. 它的产物可以拿给别人看。 这是本手册十个项目里，唯一一个能直接「展示」的作品。 而这，正是「产品落地」最朴素的定义。

常见陷阱

- **BMP 每一行的字节数必须是 4 的倍数。** 480 刚好是 4 的倍数，所以看起来没问题； 但你一旦把宽度改成 481，图片就会歪斜——因为少补了那几个填充字节。**这是格式规范，不是可以省略的优化。**
- **BMP 的像素是自下而上存储的。** 忘了这件事，得到的图片会上下颠倒。 写循环时那个 `for (int y = h - 1; y >= 0; --y)` 就是干这个的。
- 8 位灰度图必须带那 256 级调色板。漏掉它，图片查看器会打不开或者显示成乱码。
- 迭代上限 `MAX_ITER` 太小，图形边缘会很粗糙；太大则计算变慢。 **这又是一次「精度与速度」的权衡，**和项目 08 里调学习率是同一类问题。
- 直接开 `-O0` 编译并跑 480×320，你会明显感觉到等待。 **这可能是你第一次真切地意识到：算法一样，快慢可以差很多倍。**

进阶挑战

- **放大一个区域。** 把复平面的范围改成 `[-0.75, -0.73] × [0.09, 0.11]`， 并把迭代上限提到 1000。你会亲眼看到**自相似性**——一个局部里藏着整个图形的样子。
- 把 `c` 固定、把初始的 `z` 设为每个像素的坐标，得到的是**朱利亚集合**。 只改两行代码，又是一整个家族的新图形。
- 既然要写图片了，那就试试 24 位彩色 BMP：调色板那一段去掉，每个像素写三个字节。 **然后按逃逸时间给不同的通道不同的强度，做一张有颜色的图**——你会发现颜色的数学和形状的数学一样有趣。
- 用 Python 三行代码读一下你生成的这个文件：`import imageio; img = imageio.imread('mandelbrot.bmp'); imageio.imwrite('m.png', img)` 。 **这是你的 C++ 与未来的 Python 第一次握手的时刻。**
- 把这张图接到项目 03 的网格上：读取图片的像素亮度，用不同的字符打印出来。 **两个项目一拼接，你就得到了一个「终端图片查看器」。**

APPENDIX A

让代码优雅的十条习惯

这是整本手册里唯一不讲新知识的一节，但也可能是最值钱的一节。**下面十条，每一条都能在你写的每一个程序里兑现。**

#	习惯	为什么
1	**名字要能被读出来**	`averageOf` 比 `avg` 好，`avg` 比 `a` 好。**代码是写给人读的，只是顺便让机器执行。**变量名多打三个字母，未来能省半小时。
2	**一个函数只做一件事**	如果你需要用「和」来介绍一个函数（「这个函数读取并排序并打印」），它就该被拆成三个。**能一句话说清楚的函数，才配拥有一个好名字。**
3	**能加 `const` 就加 `const`**	`const` 是写给编译器看的承诺，它会在你违约时报错。**把「我不会改这个数据」变成编译期能检查的事实，等于免费雇了一个审查员。**
4	**数据与逻辑分开**	结构体只装数据，函数只做动作。项目 01 到 10 全都遵循这一条。**分开之后，换数据源不用改逻辑，改逻辑不用动数据。**
5	**不要重复自己**	当你准备第三次复制粘贴同一段代码时，停下来，把它抽成一个函数。**三次是经验法则：第一次可以随便写，第二次忍一忍，第三次必须重构。**
6	**边界要挡住**	空容器、越界下标、除以零、打不开的文件、维度不匹配的矩阵。**程序崩溃的地方，九成在边界上。**
7	**出错要出声**	返回 `false` 或者打一行 `std::cerr`，比默默算出一个错误答案好一万倍。**沉默的 bug 是最贵的 bug。**
8	**让代码的形状对应问题的形状**	项目 05 里，加减法调用乘除法，所以乘除先算——**优先级被写进了代码的结构里。**项目 03 里，八个方向变成了一张表。当代码的结构和问题的结构对上时，读代码就不需要注释了。
9	**注释写「为什么」，不写「是什么」**	`// i 加一` 这种注释是噪音；`// 每行补齐到 4 字节，这是 BMP 的格式要求` 才是真注释。**代码已经说了「是什么」，注释要说的是代码说不出的那部分。**
10	**永远带着 `-Wall` 编译**	编译器免费的警告，往往指向一个你还没遇到的 bug。**一个零警告的程序，不一定是对的；但一个满是警告的程序，大概率有问题。**

> 这十条里没有一条是「聪明」的技巧，全都是「克制」的功夫。
>
> **写代码的功力，到最后拼的不是你知道多少花招，而是你能忍住不写多少花招。**

APPENDIX B

编译、调试与求助

B.1 最常用的几条命令

命令	作用
`g++ -std=c++14 -Wall main.cpp -o main`	最常用的编译方式，本手册全程使用
`g++ -std=c++14 -Wall -O2 main.cpp -o main`	打开优化。项目 10 这种计算量大的程序会快很多
`g++ -std=c++14 -Wall -g main.cpp -o main`	带上调试信息，配合 gdb 使用
`./main` / `main.exe`	运行。Windows 下是后者，Linux / macOS / Git Bash 下是前者
`./main < input.txt`	把文件当成键盘输入喂给程序，方便反复测试
`./main > output.txt`	把输出存进文件，方便对比和粘贴
`g++ -std=c++14 -Wall -fsyntax-only main.cpp`	只检查语法，不生成可执行文件，改代码时用得很顺手

B.2 八个最常见的错误信息，以及它们的真实含义

编译器说	翻译成人话
`expected ';' before '}' token`	上一行末尾漏了分号。**看报错行号的上一行**。
`'xxx' was not declared in this scope`	这个名字你没定义、或者拼错了、或者忘了 `#include` 对应的头文件。
`undefined reference to 'yyy'`	编译通过了但链接失败。函数只有声明没有定义，或者多个文件没一起编译。
`no matching function for call to ...`	参数类型或个数对不上。检查你有没有传错顺序、或者该传引用却传了值。
`passing 'const X' as 'this' argument discards qualifiers`	你在一个 const 对象上调用了非 const 成员函数。**给那个成员函数加个 const 就好了。**
`comparison between signed and unsigned integer expressions`	警告，不是错误。`int` 和 `size()` 比较了，加个 `static_cast<int>`。
`Segmentation fault` / 程序直接闪退	访问了不属于你的内存：下标越界、用了空指针、返回了局部变量的引用。
`ld returned 1 exit status`	链接失败。Windows 上最常见的原因：**上一次运行的程序还开着**，文件被占用，关掉它再编

编译器说	翻译成人话
	译。

B.3 调试三招

Lv1

打印出来看

在关键位置加一行 `std::cerr << "i = " << i << "\n";`。 用 `cerr` 而不是 `cout`，这样即使你用 `>` 重定向了输出， 调试信息依然会显示在屏幕上。

这是最原始也最有效的办法，专业程序员每天都在用。

Lv2

把问题缩小

程序在 100 个数据上错了，就改成 3 个数据，手工把每一步的结果算一遍，和程序的输出逐个对比。

能稳定复现的小例子，比一百行日志管用。

Lv3

怀疑「刚才改的那一行」

如果代码五分钟前还是好的，那么错误几乎一定在你刚才的改动里。

先回退，再一位一位地改回来，错误就会自己冒出来。

卡住的时候，这样求助

不管是问同学、问老师还是问 AI，一段好的提问应该包含四样东西：

1. **我想做什么**（一句话说清目标）
2. **我写了什么**（贴出相关的最小代码片段，不要贴整个文件）
3. **发生了什么**（完整的报错信息，包括行号；或实际输出与预期输出的对比）
4. **我试过什么**（这一步最能让对方认真回答你）

把问题描述清楚的过程，本身就是一半的解题过程。很多时候，你在写第 3 条的时候就已经找到答案了。

APPENDIX C

从这里走向 Python 与人工智能

C.1 先说一个可能让你意外的事实

你说你之后的学习重点是 Python，而写这份手册用的是 C++。这两件事看起来有点别扭，其实一点都不矛盾——

> **你在这十个项目里真正学到的东西，几乎全都不是 C++ 的。**
> 「把数据和对数据的操作分开」「一个函数只做一件事」「用方向数组消灭重复的 if」「先取对数再求和」「训练和预测要分开」——这些是**编程的思想**，换到 Python、换到任何语言，它们一个字都不用改。

语言会变，工具会变，但你在这十个项目里建立起来的**「面对一个空白文件时知道从哪下手的底气」**，是会一直带着走的。

C.2 你现在手里有什么

能力	它是从哪来的	在 Python 里怎么用
用合适的数据结构装数据	项目 01–03 的 vector / map / 二维数组	list / dict / 嵌套 list，一一对应
把大问题拆成小函数	每一个项目	def 函数，思路完全一样
经典算法的骨架	BFS、递归下降、极小极大	Python 写起来更短，但算法一模一样
读得懂报错信息	附录 B 和你的每一次编译失败	Python 的 traceback 更好读，你已经不怕了
数值稳定性的直觉	项目 09 的取对数与平滑	numpy 里同样会遇到下溢、溢出、除零
机器学习的完整流程感	项目 08 与 09	造数据 → 训练 → 评估 → 保存，PyTorch 也是这个形状

C.3 如果还想再深入 C++

按你的专业方向，我不建议你现在花太多时间在 C++ 的高级特性上。但有三个方向值得留意：

- **STL 算法库**（`<algorithm>`）：`find_if`、`count_if`、`transform`、`accumulate`。 它们能让你的代码再短一半，而且和你将来要用的 pandas 风格非常接近。
- **把程序拆成多个文件**：`.h` 放声明、`.cpp` 放实现。 项目 07 的进阶挑战里提过。这是从「写脚本」走向「做工程」的第一步。
- **性能意识**：同一段算法，写法的不同可以让它快十倍。项目 10 的 `-O2` 和项目 06 的 alpha-beta 剪枝， 已经让你摸到这件事了。人工智能里，这一条非常值钱。

至于模板元编程、移动语义、智能指针、并发、C++20 的新特性——**先放着。等你真的遇到需要它们的问题时再学，效率会高十倍。**

C.4 给大一人工智能专业学生的一条路线

Lv1

数学，现在就开始，一天都不要等

线性代数、概率论与数理统计、微积分。

人工智能的本质是这三门数学，编程只是把它们变成可执行的形式。

项目 07 的矩阵乘法就是线性代数，项目 09 的贝叶斯就是概率论，项目 08 的「错一点改一点」就是微积分里的梯度。 数学课上听过的每一个概念，你都应该想一想：这个东西用代码怎么写？

Lv2

数据结构与算法

你在这本手册里已经摸到了队列、搜索、递归、哈希表。接下来系统地学一遍：链表、栈、树、堆、图、动态规划。

这是你和「只会调库的人」之间的差距所在。

Lv3

Python 与它的科学计算栈

语法一周就能学完（你已经有编程基础了，会非常快）。真正要花时间的是三样东西：

numpy

（把项目 07 的矩阵类换成它，速度差几百倍）、

pandas

（把项目 02 的统计换成它，三行顶你一百行）、

matplotlib

（把项目 10 的位图换成它，一行出图）。

一个高效的学法：把这本手册里的项目，用 Python 重写一遍。

你已经知道正确答案是什么了，所以可以全力关注「这门新语言是怎么表达的」。 词频统计、矩阵乘法、感知机、朴素贝叶斯——四个项目重写下来，Python 你就掌握了。

Lv4

然后是 PyTorch，以及真正的机器学习

到那个时候你会发现，

项目 08 里那三个 double 的感知机，和 PyTorch 里的一个大模型，骨架是一样的： 定义模型、前向计算、算损失、反向传播、更新参数。

你只是把参数从三个变成了几十亿个。

最后一句话

这本手册的十个项目，没有一个是为了考试准备的。它们全部都在做同一件事： **让你在「想法」和「能跑起来的东西」之间，建立一条通畅的路。**

这条路修好之后，你学什么语言、做什么方向，都只是换一辆车而已。

Ways to cpp • C++ 项目式训练手册

全部源码位于 `ways-to-cpp-code/`，每个文件都可以用 `g++ -std=c++14 -Wall <文件名> -o <输出名>` 直接编译运行。

愿你早日写出让自己觉得「这段代码真好看」的程序。